REPLICATION KIT

Jobber → Google Calendar Sync

Near real-time Jobber Job Visits on each estimator's Google Calendar, for Routemize availability and job stops.

1. Requirements: accounts, resources, settings, secrets, cost
2. Replication Guide: phase-by-phase setup with verify checks
3. Operations & Troubleshooting: daily tasks, fixes, behaviour
4. Agent Kickoff Prompt: paste into a new Claude agent
5. Code Guide: every file, request flows, Cloudflare and GitHub wiring
6. Source Code Appendix: the complete deployable code

Reference build: Cloudflare Worker `jobber-gcal-sync` · Repo `jose24xhr-sys/jobber-gcal-sync`
Prepared October 17, 2026

Jobber → Google Calendar Sync — Requirements

What this builds: a Cloudflare Worker that copies every *scheduled Jobber Job Visit* into the assigned estimator's Google Calendar within seconds. Routemize reads those calendars as availability, and a job stop can be created from each event.

Scope: Job Visits only. Requests, Quotes/Estimates, Assessments, Tasks and Invoices never create calendar events.

---

1. Accounts and access

#	Account	Who needs it	What it's used for	Notes
1	**GitHub**	Implementer	Hosts the code; pushes trigger deploys	Private repo. The Claude GitHub app must be installed on the repo if an agent pushes code
2	**Cloudflare**	Implementer	Worker, D1 database, Queues, secrets, logs	The free plan works: Queues include 10,000 ops/day and logs 200K events/day
3	**Cloudflare ↔ GitHub link**	Implementer	Workers Builds auto-deploys from the repo	The Cloudflare account must be linked to the **same** GitHub account that owns the repo
4	**Jobber Developer Center** (developer.getjobber.com)	Implementer	Creates the Jobber app: client ID/secret, scopes, webhooks	A **separate login** from the normal Jobber app
5	**Jobber account** (the customer's)	Jobber **admin** user	Approves the app once ("Allow Access")	A trial account works for testing
6	**Google Cloud** (console.cloud.google.com)	Implementer	Project, Calendar API, service account and JSON key	No billing needed for the Calendar API at this volume
7	**Google Calendar** for each estimator	Each estimator, or their Workspace admin	Shares their calendar with the service account	Only the calendar owner or a Workspace admin can share it
8	**Routemize**	Estimators / admin	Connects each estimator's Google Calendar	Uses the same calendar that receives the Jobber events

2. Tools

- **Browser access** to all of the above. An agent can do almost everything through Claude in Chrome. Exceptions: sign-ins, pasting secrets, approving OAuth screens, and downloading the Google key file are done by the human.
- **Node.js 20+ and npm**, only to run the tests or deploy from a terminal. The normal path deploys from GitHub with no local terminal.
- **Optional:** the Cloudflare MCP connector (D1 queries, creating databases). The repo `jose24xhr-sys/jobber-gcal-sync` holds the complete, tested code: 65 tests.

3. Cloudflare resources

Resource	Name	Binding	Created by
Worker	`jobber-gcal-sync`	—	Workers Builds (import repo)
D1 database	`jobber_gcal_sync`	`DB`	Dashboard or `wrangler d1 create`; then run `migrations/0001_init.sql`
Queue	`jobber-visit-sync`	`SYNC_QUEUE` (producer + consumer)	Dashboard → Queues
Queue (dead letter)	`jobber-visit-sync-dlq`	—	Dashboard → Queues
Cron	daily `17 4 * * *`	—	`wrangler.toml` (prunes old logs)

4. Configuration values

Plain variables (in `wrangler.toml`, safe to commit)

Name	Example	Purpose
`JOBBER_API_VERSION`	`2026-09-25`	Required `X-JOBBER-GRAPHQL-VERSION` header
`JOBBER_REDIRECT_URI`	`https://jobber-gcal-sync.<subdomain>.workers.dev/oauth/jobber/callback`	Must match the Jobber app's Callback URL exactly
`GOOGLE_AUTH_MODE`	`service_account`	Or `oauth` (not recommended)
`DEFAULT_TIMEZONE`	`America/New_York`	Used only for all-day visits
`ALLOWED_TOPICS`	`VISIT_CREATE,VISIT_UPDATE,VISIT_COMPLETE,VISIT_DESTROY,JOB_CREATE,JOB_UPDATE,JOB_DESTROY,APP_DISCONNECT`	Webhook topics acted on
`EVENT_TITLE_PREFIX`	*(empty)* or `[Jobber]`	Optional prefix on event titles

Name	Example	Purpose
`JOBBER_CLIENT_ID`	from the Jobber app	Public identifier
`GOOGLE_SA_CLIENT_EMAIL`	`jobber-sync@<project>.iam.gserviceaccount.com`	Service account identity
`keep_vars = true`	top of `wrangler.toml`	Stops Git deploys from deleting variables added in the dashboard

Secrets (Cloudflare → Worker → Settings → Variables and Secrets → type Secret)

Name	Where it comes from	Format
`JOBBER_CLIENT_SECRET`	Jobber app → Authorization section	As shown. It also verifies webhook signatures
`GOOGLE_SA_PRIVATE_KEY`	The service-account JSON key file	Paste the **one-line** `"private_key": "-----BEGIN…\n…-----END PRIVATE KEY-----\n"` text from the JSON. **Do not paste the multi-line PEM**: the dashboard field keeps only the first line
`TOKEN_ENCRYPTION_KEY`	You generate it	Ideally `openssl rand -base64 32`; any long passphrase also works
`ADMIN_API_TOKEN`	You generate it	Long random string; it protects `/admin` and `/oauth/jobber/start`

Never commit secrets, put them in URLs, or paste them into chat.

5. Jobber app settings

Setting	Value
App name / developer	e.g. "Routemize Calendar Sync" / "Routemize"
Callback URL	`https://jobber-gcal-sync.<subdomain>.workers.dev/oauth/jobber/callback`
Short description	Required to save
Scopes (read only)	**Clients**, **Jobs**, **Scheduled Items**, **Users** — no Write boxes
Webhooks (all → `https://jobber-gcal-sync.<subdomain>.workers.dev/webhooks/jobber`)	`VISIT_CREATE`, `VISIT_UPDATE`, `VISIT_COMPLETE`, `VISIT_DESTROY`, `JOB_CREATE`, `JOB_UPDATE`, `JOB_DESTROY`, `APP_DISCONNECT`
Refresh token rotation	**ON** (default)

6. Google settings

Item	Value
Project	e.g. `jobber-gcal-sync`
API enabled	Google Calendar API (`calendar-json.googleapis.com`)
Service account	e.g. `jobber-sync` — **no project roles**
Key	JSON (Keys → Add key → Create new key → JSON)
OAuth scope used	`https://www.googleapis.com/auth/calendar.events` only
Per-estimator calendar sharing	Share with the service account email → **"Make changes (see private events as free/busy)"**

7. Information to collect from the business

- ☐ List of estimators/techs who should get Jobber visits on a calendar
- ☐ For each: **Jobber user** (name) → **Google Calendar ID**, usually their email; it's under Calendar settings → Integrate calendar
- ☐ Confirmation that each estimator has shared their calendar with the service account
- ☐ Business timezone (for all-day visits)
- ☐ Which Jobber admin will click "Allow Access"

8. Hosting model and cost

Two ways to offer it to other businesses

Model	How it works	Limits
Hosted (one Worker, many businesses)	Each business's Jobber admin opens `/oauth/jobber/start` and clicks Allow Access. Their estimators share calendars with your service account. The code is already multi-account	A custom Jobber app can connect to **5 paying Jobber accounts** at most; beyond that it must be published in the Jobber App Marketplace (review, logo, Terms of Service and Privacy Policy URLs). You hold customers' tokens and client data, so have a privacy policy
Self-hosted (one copy per business)	The business follows this guide with its own Cloudflare, Google and Jobber app	No 5-account limit; costs and data stay with the business

Running cost (prices checked Oct 2026)

Service	Cost	Notes
Cloudflare	**$0** on the free plan	Free plan: 100,000 Worker requests/day, 10,000 Queue operations/day, D1 100,000 rows written/day + 5 GB. One visit change ≈ 2 webhooks ≈ 2 requests + ~6 queue ops, so about 1,500 visit changes/day fit free. Paid: **$5/month** (10M requests + 1M queue ops/month included)

Service	Cost	Notes
Google Calendar API	**$0**	No per-call charge; daily quotas are far above this volume. No billing account required
Claude	**$0 to run**	The Worker never calls Claude; Claude was only used to build it
Jobber	**$0** for API use	The business needs its normal Jobber subscription

Jobber → Google Calendar Sync — Replication Guide

A step-by-step playbook for a person or a Claude agent to stand up a fresh copy of this integration for a new business. Every phase ends with a **Verify** check, so don't start a phase until the one before it passes.

Legend: AGENT = an agent can do it (browser or connector) · HUMAN = the human must do it (sign-in, secret, approval, download)

```
Jobber (visit created/edited/deleted)
    │  webhook: topic + IDs only, HMAC-signed
    ▼
Cloudflare Worker  /webhooks/jobber ── verify signature ── enqueue ── 200 OK (< 1 s)
    │
    ▼  Queue consumer (retries, dead-letter queue)
Jobber GraphQL: visit(id) + job + client + property + assignedUsers
    │  validate: Visit on a Job, scheduled, assigned
    ▼
Google Calendar API (service account) → event on each assignee's calendar
    │
    ▼
Routemize reads the estimator's calendar → availability blocked → job stop can be created
```

Phase 0 — Collect inputs HUMAN

Use the checklist in `01-REQUIREMENTS.md §7`. Decide names now:

Placeholder	Example used in the reference build
`<GITHUB_OWNER>/<REPO>`	`jose24xhr-sys/jobber-gcal-sync`
`<CF_SUBDOMAIN>`	`joseramirez` (from Cloudflare → Workers → Subdomain)
`<WORKER_URL>`	`https://jobber-gcal-sync.<CF_SUBDOMAIN>.workers.dev`
`<GCP_PROJECT>`	`jobber-gcal-sync`
`<SA_EMAIL>`	`jobber-sync@<GCP_PROJECT>.iam.gserviceaccount.com`

Agent rules: never type passwords, API keys or secrets into any field, and never put them in URLs or chat. Ask the human to paste them. Ask before any OAuth/app-install approval, calendar sharing, or file download.

Phase 1 — Code repository

1. AGENT Create a **private** repo `<REPO>` (GitHub → New repository; no README/gitignore/license).
2. AGENT Copy the reference code into it. Either copy the source repo, or push the `jobber-gcal-sync.zip` contents. If an agent pushes from its own workspace, its GitHub app must have access. The safe way: install the **Claude** GitHub app with *Only select repositories* → *`<REPO>`* (HUMAN approves).
3. AGENT Keep `wrangler.toml` placeholders for now; Phase 3 fills them.

Verify: the repo shows `src/`, `test/`, `migrations/`, `wrangler.toml`, `README.md`. Optional: `npm ci && npm test` → **65 passed**.

Phase 2 — Cloudflare data resources

1. AGENT Pick the right Cloudflare account. If there are several, use the one the business wants to own the Worker.
2. AGENT **D1:** create database `jobber_gcal_sync` (location: closest to the business, e.g. ENAM). Copy its **database ID**.
3. AGENT **Schema:** run all of `migrations/0001_init.sql` against it, through the D1 Console in the dashboard or the Cloudflare connector's `d1_database_query`. It creates 5 tables: `jobber_accounts`, `oauth_states`, `user_calendar_map`, `visit_events`, `sync_log`.
4. AGENT **Queues:** create `jobber-visit-sync` and `jobber-visit-sync-dlq` (Workers → Queues → Create Queue; jurisdiction None).
5. AGENT Read the account's workers.dev subdomain (Workers → *Subdomain* page). **Do not change it**: that would move every Worker on the account.

Verify: D1 lists 5 tables; Queues lists 2 queues (status "Inactive" is normal until deploy).

Phase 3 — Configure and deploy the Worker

1. AGENT Edit `wrangler.toml`:
   - `database_id = "<D1 ID>"`
   - `JOBBER_REDIRECT_URI = "<WORKER_URL>/oauth/jobber/callback"`
   - `DEFAULT_TIMEZONE` = the business timezone
   - keep `keep_vars = true` at the top
   - Commit and push.
2. AGENT Cloudflare → Workers & Pages → **Create** → **Continue with GitHub** → pick `<REPO>` → deploy. Defaults are fine: deploy command `npx wrangler deploy`, root `/`.
   - If the repo doesn't appear, the Cloudflare account is linked to a different GitHub account. Choose **Add GitHub account** and HUMAN authorizes Cloudflare's GitHub app for `<GITHUB_OWNER>`. The approval opens in a pop-up the agent can't see, so ask the human to finish it.
3. AGENT Wait for the build: Initializing → Cloning → Installing → Deploying, all green.

Verify: `GET <WORKER_URL>/health` → `ok`.

From now on, **every push to `main` redeploys automatically**, in about 1 minute.

Phase 4 — Jobber app

1. HUMAN Log in at developer.getjobber.com (separate from the normal Jobber login).
2. AGENT Manage Apps → **New**. Fill in:
   - App name, Developer name, **Callback URL** `<WORKER_URL>/oauth/jobber/callback`, **Short description** (required).
   - **Scopes**: toggle ON **Clients**, **Jobs**, **Scheduled Items**, **Users**. Leave **Write** unchecked.
   - **Webhooks**: add 8 rows, all with URL `<WORKER_URL>/webhooks/jobber`: `VISIT_CREATE`, `VISIT_UPDATE`, `VISIT_COMPLETE`, `VISIT_DESTROY`, `JOB_CREATE`, `JOB_UPDATE`, `JOB_DESTROY`, `APP_DISCONNECT`.
     - NOTE The topic picker is a type-ahead. **Click the option**; pressing Enter selects the first item in the full list (`APP_CONNECT`).
   - Refresh token rotation: leave **ON**.
   - **Save App**.
3. AGENT Read back all 8 topic/URL pairs (e.g. from the form's input values) before saving.

Verify: "App updated" toast. The Authorization section shows Client ID and Client Secret.

Phase 5 — Google Cloud

1. AGENT console.cloud.google.com → New project `<GCP_PROJECT>` (no organization is fine).
2. AGENT APIs & Services → Library → **Google Calendar API** → **Enable**.
3. AGENT IAM & Admin → Service accounts → **Create**: name `jobber-sync`. **Skip roles** and close the role picker; none are needed.
4. HUMAN Service account → **Keys** → Add key → Create new key → **JSON**. Downloads a file; keep it private.
   - If key creation is blocked by an org policy (`iam.disableServiceAccountKeyCreation`), a Google admin must allow it for this project.

Verify: the API page shows **Status: Enabled**; the service accounts list shows `<SA_EMAIL>`.

Phase 6 — Secrets and variables

HUMAN Cloudflare → Workers & Pages → `jobber-gcal-sync` → **Settings → Variables and Secrets → + Add** (type **Secret** for all four):

Name	Value
`JOBBER_CLIENT_SECRET`	From the Jobber app

Name	Value
`GOOGLE_SA_PRIVATE_KEY`	In the JSON file, the single line `"private_key": "-----BEGIN PRIVATE KEY-----\nMII…\n-----END PRIVATE KEY-----\n",` copied **whole**. Extra quotes or the label are fine; the code cleans them. Never paste the multi-line PEM: the field keeps only the first 64 characters
`TOKEN_ENCRYPTION_KEY`	Long random value. **Set it once, before connecting Jobber.** Changing it later means reconnecting
`ADMIN_API_TOKEN`	Long random value; save it in a password manager

AGENT Put `JOBBER_CLIENT_ID` and `GOOGLE_SA_CLIENT_EMAIL` in `wrangler.toml` `[vars]` (not secret) and push.

Verify (agent): the Settings page lists all 6 names. `TOKEN_ENCRYPTION_KEY`, `JOBBER_CLIENT_SECRET`, `GOOGLE_SA_PRIVATE_KEY` and `ADMIN_API_TOKEN` must show **"Value encrypted"**. If any shows a plain value, ask the human to edit it and switch the type to **Secret**.

Phase 7 — Connect Jobber

1. **HUMAN** Open `<WORKER_URL>/oauth/jobber/start` → paste `ADMIN_API_TOKEN` in the password box → **Continue**.
2. **HUMAN** In Jobber's consent screen, as a Jobber **admin**: **Allow Access**.
3. The success page shows **"Connected Jobber account `<name>`"** and the **Account ID** (e.g. `Z2lkOi8vSm9iYmVyL0FjY291bnQv…`).

Verify (agent, via D1):

```
SELECT account_id, account_name, status FROM jobber_accounts;   -- status = active
```

Phase 8 — Share calendars and map estimators

1. **HUMAN** Each estimator, or the Workspace admin, opens Google Calendar → Settings → *their calendar* → **Shared with → Add people and groups** → `<SA_EMAIL>` → **Make changes (see private events as free/busy)** → Send.
   - An agent can do this only for calendars owned by the Google account signed in to the browser, and only after the human agrees.
2. **AGENT** / **HUMAN** Open `<WORKER_URL>/admin` → enter `ADMIN_API_TOKEN`. The page lists every Jobber team member. Type each estimator's **Calendar ID** (usually their email) → **Save mappings**. Leave a row blank to not sync that person.
   - Alternative (agent, no token needed): insert into D1 directly. The API user ID = base64 of `gid://Jobber/User/<n>`. `<n>` comes from Jobber web → Settings → Manage Team, whose link `/manage_team/<base64 n>` holds the number base64-encoded.

```
INSERT INTO user_calendar_map (account_id, jobber_user_id, google_calendar_id, label, active,
updated_at)
VALUES ('<ACCOUNT_ID>', '<USER_ENCODED_ID>', 'estimator@company.com', 'Estimator name', 1,
strftime('%s','now')*1000)
ON CONFLICT(account_id, jobber_user_id) DO UPDATE SET google_calendar_id =
excluded.google_calendar_id, active = 1;
```

3. HUMAN Connect the same calendars in **Routemize** for each estimator, if they aren't already.

Verify: `/admin` shows the calendar next to each estimator.

Phase 9 — End-to-end test

1. AGENT In Jobber: **Create → Job** for an existing client, title `TEST - Calendar sync check`, one visit today or tomorrow with start and end times, **assigned** to a mapped estimator, instructions "Safe to delete" → **Save Job**.
2. AGENT Within ~10 s, `/admin` → *Recent activity* shows `created VISIT_CREATE`. A `noop` right after is normal: Jobber sends a second webhook, and it's de-duplicated.
3. AGENT Google Calendar (that estimator) shows "**`<Client> - <Job title>`**" at the right time, with the property address as location and client, job #, instructions and the Jobber link in the description.
4. Then test the rest:

Do in Jobber	Expect in Google Calendar
Reschedule the visit	Same event moves (no duplicate)
Reassign to another mapped person	Event leaves calendar A, appears on calendar B
Assign two people	One event on each calendar
Unschedule the visit	Event removed
Delete the visit / job	Event removed
Create a Request or Quote	Nothing

5. HUMAN In Routemize: the slot is blocked and a job stop can be created from the event.
6. AGENT Delete the TEST job when done; its event is removed automatically.

If nothing appears: see `03-OPERATIONS.md → Troubleshooting`. The Cloudflare **Observability → Events** log shows the exact error per message.

Done criteria

- ☐ `/health` returns `ok`
- ☐ 4 secrets show "Value encrypted"; `keep_vars = true` is in `wrangler.toml`
- ☐ `jobber_accounts.status = active`
- ☐ Every estimator is mapped on `/admin`, and their calendar is shared with `<SA_EMAIL>`

- ☐ Create, reschedule, reassign and delete all behave as in the Phase 9 table
- ☐ The Requests/Quotes test produces no events
- ☐ TEST job deleted

Jobber → Google Calendar Sync — Operations & Troubleshooting

Day-to-day tasks

Add a new estimator

1. In Jobber, add them as a team member (Settings → Manage Team).
2. They share their Google Calendar with `<SA_EMAIL>` using **Make changes (see private events as free/busy)**.
3. `<WORKER_URL>/admin` → enter the admin token → type their Calendar ID next to their name → **Save mappings**.
4. Connect the same calendar in Routemize.

Their visits sync from the next change onward. To backfill an existing visit, edit it in Jobber (any save fires a webhook).

Remove or pause an estimator

On `/admin`, clear their Calendar ID and save. Events already on their calendar stay until each visit changes or is deleted. Then remove the calendar share.

Move an estimator to a new calendar

Change the Calendar ID on `/admin`. On each visit's next change, the event moves from the old calendar to the new one.

Where to look

Need	Place
Recent syncs per visit (created/updated/deleted/skipped/error)	`<WORKER_URL>/admin` → Recent activity, or D1 `sync_log`
Full request/queue logs with error text	Cloudflare → Worker → **Observability → Events**
Failed messages after 8 retries	Queue `jobber-visit-sync-dlq`
Which calendar/event a visit maps to	D1 `visit_events` (or JSON `GET /admin/visits?accountId=…&visitId=…` with `Authorization: Bearer <ADMIN_API_TOKEN>`)
Re-run one visit	JSON `POST /admin/resync {"accountId","visitId"}` with the bearer token, or just edit the visit in Jobber

Useful D1 queries

```
-- Last 20 sync actions
SELECT datetime(at/1000,'unixepoch') AS utc, topic, action, detail FROM sync_log ORDER BY id DESC
LIMIT 20;
-- Connection health
SELECT account_name, status, datetime(access_expires_at/1000,'unixepoch') AS token_expires FROM
jobber_accounts;
-- Mappings
SELECT label, jobber_user_id, google_calendar_id, active FROM user_calendar_map;
```

Troubleshooting (every row was hit or prevented during the reference build)

Symptom	Cause	Fix
`/oauth/jobber/start?key=…` says **unauthorized**	The placeholder was left in the link, or the token has `# & + $` characters that break inside URLs	Open `/oauth/jobber/start` with no `?key` and type the token into the password box
Log: `Invalid PKCS8 input`; diagnostic shows `base64Length: 64`, `hasBeginMarker: false`	The multi-line PEM was pasted into the single-line secret field, so only the first line was kept	Re-paste `GOOGLE_SA_PRIVATE_KEY` using the one-line `"private_key": "…\n…"` text from the JSON file. Queued visits sync automatically once fixed
Log: `TOKEN_ENCRYPTION_KEY must be 32 bytes` (old builds)	A passphrase was used instead of 32-byte base64	Current code accepts any passphrase. Make sure it's type **Secret**
Dashboard variables disappear after a GitHub push	Wrangler replaces variables not in `wrangler.toml`	`keep_vars = true` (already set); put non-secret values in `wrangler.toml`
Cloudflare "Import repository" doesn't list the repo	The Cloudflare account is linked to a different GitHub account	**Add GitHub account** → authorize for the repo owner (human finishes the pop-up)
Agent can't push: "repository not found / no access"	The Claude GitHub app isn't installed on that private repo	Install the Claude app → *Only select repositories* → the repo
Jobber webhook topic saved as `APP_CONNECT`	Pressed Enter in the type-ahead	Click the option instead; read back all rows before saving

Symptom	Cause	Fix
Nothing in `sync_log` and no "webhook queued" in logs	Webhooks not reaching the Worker: wrong URL, app not saved, or account not connected	Check the Jobber app's webhook URLs = `<WORKER_URL>/webhooks/jobber`; `jobber_accounts.status = active`
Log: `Rejected webhook: invalid or missing signature`	`JOBBER_CLIENT_SECRET` doesn't match the app (or the secret was regenerated)	Re-paste the secret from the Jobber app
`sync_log`: `No calendar mapping for Jobber user …`	The assignee isn't mapped	Map them on `/admin`
Error: Google Calendar **403/404** on a calendar ID	That calendar isn't shared with `<SA_EMAIL>`, or the ID is wrong	Share with "Make changes…"; check the Calendar ID under Settings → Integrate calendar
`jobber_accounts.status = reauth_required`	Refresh token invalid (app disconnected, secret regenerated, scopes changed)	Re-run `/oauth/jobber/start`
Two identical events on one calendar	Two assignees are mapped to the **same** calendar (normal in testing)	Map each person to their own calendar
All-day visit on the wrong date	`DEFAULT_TIMEZONE` doesn't match the business	Set it in `wrangler.toml` and push
`Jobber API version warning` in logs	The pinned `JOBBER_API_VERSION` is nearing end of support	Check developer.getjobber.com/docs/changelog, bump the version, push

Behaviour reference

- **Idempotent:** the event ID is derived from (account, visit, user). Repeated or duplicate webhooks never create duplicates. An unchanged visit causes zero Google writes (`noop`).
- **New jobs:** `JOB_CREATE` fetches every visit of the new job (up to 100) and syncs each one. Jobber also sends `VISIT_CREATE` per visit, so this is a safety net, and the duplicates are no-ops.
- **Deletes survive the visit being gone:** the visit→event mapping is stored before the event is created.
- **Retries:** network errors, 429 and 5xx are retried inline, then by the queue with backoff (15 s → 15 min, 8 attempts), then sent to the DLQ. Config errors that won't fix themselves (calendar not shared) are logged and not retried. A bad Google key **is** retried, so visits sync once the key is fixed.

- **Token safety:** Jobber refresh-token rotation is handled with a single-refresher lock. Tokens are AES-GCM encrypted in D1 and never logged.
- **Events:** busy (opaque), no reminders, no invitation emails. Title = Jobber visit title (Jobber defaults it to "Client - Job title"). Location = full property address. Description = client, job #, instructions, Jobber link.

Verified live in the reference build (Oct 17, 2026)

- Webhook received → HMAC verified → queued → Jobber GraphQL fetch → Google event created on the mapped calendar, with correct time and address
- Duplicate follow-up webhook recognized (`noop`)
- Reschedule, reassign, unschedule, delete and Request/Quote filtering are covered by the automated test suite (65 tests); run the Phase 9 table on each new install.

Agent kickoff prompt

Paste the block below into a new Claude agent session. Attach the Replication Kit PDF (or `01`–`05` docs) and either give it access to the code repo or attach `jobber-gcal-sync-source.zip`.

```
You are setting up the "Jobber Job Visits → Google Calendar" integration for a new business.
It is a Cloudflare Worker (code is complete and tested — do not rewrite it) that mirrors
scheduled Jobber Job Visits into each assigned estimator's Google Calendar within seconds,
so Routemize sees them as availability and can create job stops from them.

Source code: <SOURCE_REPO or "the attached jobber-gcal-sync-source.zip"> (copy it into a new private
repo
for this business). 05-CODE-GUIDE.md explains every file and the Cloudflare/GitHub wiring.
Follow 02-REPLICATION-GUIDE.md phase by phase. Do not start a phase until the previous
phase's "Verify" check passes. Use 01-REQUIREMENTS.md for every name, scope, topic and
setting. Use 03-OPERATIONS.md when anything fails — check Cloudflare Observability logs
and the D1 sync_log before changing code.

Business inputs:
- Business / Jobber account name: <NAME>
- Cloudflare account to use: <CF_ACCOUNT_EMAIL>
- GitHub owner for the new repo: <GITHUB_OWNER>
- Business timezone: <IANA_TZ, e.g. America/New_York>
- Estimators (Jobber name → Google Calendar ID):
  - <Name> → <calendar@company.com>
  - <Name> → <calendar@company.com>

Rules:
- Never type passwords, API keys, client secrets, private keys or tokens into any field,
  URL or chat. Ask me to paste them. Ask me before: installing/authorizing any app,
  sharing a calendar, downloading a file, or creating records in a live Jobber account.
- Sign-ins are done by me. If a pop-up window opens, ask me to finish it.
- Jobber webhook topic picker: click the option, never press Enter.
- GOOGLE_SA_PRIVATE_KEY must be pasted as the one-line JSON "private_key" value.
- Keep keep_vars = true in wrangler.toml; non-secret values go in [vars].
- Finish with the Phase 9 test table and the "Done criteria" checklist, then report
  what passed, what is pending (e.g. estimators who haven't shared calendars), and
  delete the TEST job.
```

Jobber → Google Calendar Sync — Code Guide (Cloudflare + GitHub)

What the code does, file by file, and how it runs on Cloudflare and ships from GitHub. The full source is in **06-SOURCE-CODE** (appendix) and in `jobber-gcal-sync-source.zip`.

Stack: TypeScript on Cloudflare Workers, with no runtime dependencies (only Web APIs: `fetch`, `crypto.subtle`). Storage is Cloudflare D1 (SQLite) plus Cloudflare Queues. Tests use Vitest; the dev tools are `wrangler`, `typescript` and `vitest`.

---

1. Repository layout

```
jobber-gcal-sync/
├── wrangler.toml              Cloudflare config: bindings, vars, queue, cron
├── package.json               scripts: test, typecheck, dev, deploy, db:migrate:*
├── tsconfig.json · vitest.config.ts
├── migrations/0001_init.sql   D1 schema (5 tables)
├── src/                       the Worker (≈1,900 lines)
├── test/                      65 automated tests + fakes for Jobber/Google/D1
├── graphql/                   the exact GraphQL queries, for checking in Jobber's GraphiQL
├── scripts/                   sign-webhook.mjs (send a signed test webhook), render-docs-pdf.py
└── docs/                      these guides + PDFs
```

`src/`: what each file does

File	Responsibility	Key functions
`index.ts`	**Entry point.** HTTP router (`fetch`), queue consumer (`queue`), daily cron (`scheduled`)	`handleWebhook`, default export `{ fetch, queue, scheduled }`
`env.ts`	Types for every binding, variable and secret, plus the queue message shape	`Env`, `SyncMessage`
`config.ts`	Webhook topic allow-list and routing	`classifyTopic()` → `visit_upsert`, `visit_delete`, `job_create`, `job_update`, `job_delete`, `app_disconnect`, `ignored`
`log.ts`	JSON logs; **redacts** any field that looks like a token/secret/key	`log()`, `snippet()`
`jobber/webhook.ts`	Verifies `X-Jobber-Hmac-SHA256` and parses the payload	`verifyJobberSignature()`, `parseJobberWebhook()`
`jobber/oauth.ts`	Jobber OAuth: authorize redirect (PKCE + state) and callback/code exchange	`startJobberOAuth()`, `handleJobberCallback()`

File	Responsibility	Key functions
`jobber/tokens.ts`	Encrypted token storage; **rotation-safe refresh** with a D1 lock	`getJobberAccessToken()`, `storeTokens()`
`jobber/graphql.ts`	GraphQL client (version header, 401 → refresh, THROTTLED → retry) and the queries	`jobberGraphQL()`, `VISIT_QUERY`, `JOB_VISITS_QUERY`, `USERS_QUERY`, `ACCOUNT_QUERY`
`jobber/ids.ts`	Decodes Jobber EncodedIds (`gid://Jobber/<Type>/<n>`) to reject non-Visit objects	`decodeEncodedId()`, `isDefinitelyNot()`
`google/auth.ts`	Google access token: service-account JWT (RS256) or OAuth refresh; accepts the private key in any paste format	`getGoogleAccessToken()`, `buildServiceAccountJwt()`, `normalizePrivateKey()`
`google/calendar.ts`	Idempotent event **upsert** (PUT → POST → PUT) and **delete** (404/410 = already gone)	`upsertCalendarEvent()`, `deleteCalendarEvent()`
`sync/validate.ts`	The gate before every Google write: Visit? on a Job? scheduled? assigned?	`validateJobVisit()`
`sync/mapEvent.ts`	Jobber visit → Google event body; deterministic event ID; content hash	`buildEventBody()`, `deterministicEventId()`, `contentHash()`
`sync/processor.ts`	**The sync brain:** per topic, decides create/update/move/delete	`processMessage()`, `syncVisit()`, `removeVisit()`
`db/repo.ts`	Every D1 query	accounts, OAuth state, mappings, visit events, sync log
`admin.ts`	JSON admin API (Bearer `ADMIN_API_TOKEN`)	`handleAdmin()`
`adminUi.ts`	Browser pages with a password box: connect Jobber, map team → calendars, recent activity	`handleJobberStartPage()`, `handleAdminPage()`
`util/crypto.ts`	base64/base64url, SHA-256, HMAC, PKCE, constant-time compare, AES-GCM	`encryptString()`, `decryptString()`
`util/retry.ts`	`fetchWithRetry()` (backoff, Retry-After), `RetryableError` vs `PermanentError`, queue backoff	`queueBackoffSeconds()`

2. How a request flows

A. Jobber webhook → calendar (the main path)

```
POST /webhooks/jobber                                    index.ts › handleWebhook
  1. read raw body
  2. verifyJobberSignature(raw, X-Jobber-Hmac-SHA256, JOBBER_CLIENT_SECRET)   → 401 if bad
  3. parseJobberWebhook → {topic, accountId, itemId}
  4. classifyTopic → "ignored"? → 200 and log "Skipped non-job event"
  5. SYNC_QUEUE.send({topic, accountId, itemId})          → 200 "ok" (well under 1 s)

Queue consumer                                           index.ts › queue
  - collapses duplicates in the batch (same kind + account + item)
  - processMessage(msg)                                  sync/processor.ts
      visit_upsert  → syncVisit(visitId)
      visit_delete  → removeVisit(visitId)            (uses stored mapping; no Jobber call)
      job_create    → JOB_VISITS_QUERY → syncVisit for each visit (≤100)
      job_update    → syncVisit for each visit already tracked for that job
      job_delete    → removeVisit for each tracked visit
      app_disconnect→ account status = disconnected
  - success → ack · RetryableError → retry with backoff · PermanentError → log + ack

syncVisit(visitId)
  1. jobberGraphQL(VISIT_QUERY, {id})                    jobber/graphql.ts
  2. validateJobVisit()  → not OK? log reason; remove old events if it had some
  3. look up calendars for assigned users                db/repo.ts › getCalendarsForUsers
  4. delete events on calendars that no longer apply (reassigned/unmapped)
  5. for each mapped assignee:
       eventId = "jv" + sha256(account|visit|user)[0..40]
       body    = buildEventBody(); hash = contentHash()
       unchanged hash → "noop"
       else save mapping row FIRST → upsertCalendarEvent() → save hash
```

B. Connect Jobber (OAuth)

`GET /oauth/jobber/start` shows a password box (`adminUi.ts`). Submitting the correct `ADMIN_API_TOKEN` stores state plus a PKCE verifier in `oauth_states` and redirects to Jobber. Jobber then calls `/oauth/jobber/callback?code&state` (`oauth.ts`). The Worker checks the state, exchanges the code (with `code_verifier`), queries `account{id name}`, and stores the **encrypted** tokens in `jobber_accounts`.

C. Admin page

`GET /admin` shows the password box. `POST /admin` with the token loads Jobber users (`USERS_QUERY`) and shows a calendar field for each. Saving writes `user_calendar_map`. The page also shows the last 25 `sync_log` rows.

D. Daily cron (`17 4 * * *` UTC)

`repo.prune()` deletes OAuth states older than 1 h and `sync_log` rows older than 30 days.

---

3. Data model (D1, `migrations/0001_init.sql`)

Table	Key	Purpose
`jobber_accounts`	`account_id`	One row per connected Jobber account: AES-GCM-encrypted access/refresh tokens, expiry, refresh lock, status (`active`, `reauth_required`, `disconnected`)
`oauth_states`	`state`	Single-use OAuth state + PKCE verifier (10-min validity)
`user_calendar_map`	`(account_id, jobber_user_id)`	**Jobber user → Google Calendar ID** (platform IDs, not names)
`visit_events`	`(account_id, visit_id, jobber_user_id)`	What we wrote to Google: calendar, event ID, job ID, content hash
`sync_log`	`id`	Audit trail: created / updated / deleted / noop / skipped / error

---

4. Cloudflare: how it's wired

`wrangler.toml`, line by line

Setting	Meaning
`name = "jobber-gcal-sync"`, `main = "src/index.ts"`	Worker name (also the URL prefix) and entry file
`compatibility_date`	Runtime version the Worker is pinned to
`keep_vars = true`	Deploys **keep** variables added in the dashboard (otherwise a Git deploy deletes them)
`[observability] enabled`	Logs appear under Worker → Observability → Events
`[vars]`	Non-secret settings (API version, callback URL, timezone, topics, client ID, service-account email)
`[[d1_databases]] binding="DB"`	Code uses `env.DB`; `database_id` must be **your** D1 ID
`[[queues.producers]] binding="SYNC_QUEUE"`	Code sends with `env.SYNC_QUEUE.send()`
`[[queues.consumers]]`	The same Worker consumes: `max_batch_size=10`, `max_batch_timeout=1` s, `max_retries=8`, **`max_concurrency=1`** (one consumer at a time, so token refreshes never race), `dead_letter_queue`
`[triggers] crons`	Daily cleanup

Secrets are not in this file. They're added in the dashboard (Settings → Variables and Secrets → type **Secret**) or with `npx wrangler secret put NAME`.

Cloudflare dashboard map

Task	Where
See deploys / build logs	Workers & Pages → `jobber-gcal-sync` → **Deployments** (each push = one build)
Live logs, error details	→ **Observability → Events** (expand a row for the JSON fields)
Secrets / variables / bindings	→ **Settings → Variables and Secrets / Bindings**
Git connection, build command	→ **Settings → Build** (repo, branch `main`, deploy command `npx wrangler deploy`)
Run SQL	Storage & Databases → **D1** → `jobber_gcal_sync` → **Console**
Queue depth, DLQ	Compute → **Queues**
workers.dev URL	Worker → **Domains**, or Workers → *Subdomain* (don't change it)

Deploying without GitHub (CLI alternative)

```
npm ci
npx wrangler login
npx wrangler d1 create jobber_gcal_sync            # put the ID in wrangler.toml
npx wrangler d1 execute jobber_gcal_sync --remote --file=migrations/0001_init.sql
npx wrangler queues create jobber-visit-sync
npx wrangler queues create jobber-visit-sync-dlq
npx wrangler secret put JOBBER_CLIENT_SECRET       # repeat for the other 3 secrets
npx wrangler deploy
```

Local development

```
cp .dev.vars.example .dev.vars    # fill in values; never commit
npm run db:migrate:local
npm run dev                       # http://localhost:8787 (local D1 + local queue)
JOBBER_CLIENT_SECRET=... node scripts/sign-webhook.mjs VISIT_UPDATE <visitId> <accountId>
http://localhost:8787/webhooks/jobber
```

---

5. GitHub: how changes ship

Piece	Role
Repo (private), branch `main`	Source of truth. **`main` = production**: every push to `main` triggers a Cloudflare build and deploy (~1 min)

Piece	Role
Cloudflare Workers and Pages GitHub app	Lets Cloudflare clone the repo for builds. It must be authorized for the repo owner's account
Claude GitHub app (optional)	Lets a Claude agent push commits. Install with *Only select repositories* → *this repo*

Changing code from the GitHub website (no terminal):

1. Open the file on github.com → the pencil **Edit** icon → change it → **Commit changes** to `main`.
2. Cloudflare → Deployments: a new build appears; wait for **Ready**.
3. Check `/health` and Observability → Events.

For risky changes, commit to a branch and open a pull request. Merge into `main` only after running `npm test` (locally or in a Codespace). Pushes to `main` go live immediately.

Rolling back: Cloudflare → Deployments → pick an earlier version → **Rollback**, or revert the commit on GitHub.

Never commit: `.dev.vars`, the Google JSON key, or any token. `.gitignore` already excludes `.dev.vars`, `.wrangler/` and `node_modules/`.

---

6. Common changes: where to edit

I want to…	Edit
Change the event title format or add a prefix	`EVENT_TITLE_PREFIX` var, or `buildEventBody()` in `src/sync/mapEvent.ts`
Add more info to the event description	`buildEventBody()` (description array) + add the field to `VISIT_QUERY` in `src/jobber/graphql.ts`
Change the timezone for all-day visits	`DEFAULT_TIMEZONE` in `wrangler.toml`
React to another Jobber webhook topic	Add it to `ALLOWED_TOPICS` + `DEFAULT_ALLOWED_TOPICS` (`src/config.ts`), handle its kind in `processMessage()`, subscribe it in the Jobber app
Stop showing events as busy	`transparency` in `buildEventBody()` (`"transparent"`)
Upgrade the Jobber API version	`JOBBER_API_VERSION` in `wrangler.toml` (check the Jobber changelog first)
Change retry behaviour	`src/util/retry.ts` and `max_retries` in `wrangler.toml`

After any change: `npm run typecheck && npm test`, then push.

---

7. Tests

`npm test` runs **65 tests** in about 2 seconds, with no network or credentials. `test/helpers/` contains:

- `d1.ts` : a real SQLite database built from the actual migration file
- `fakes.ts` : in-memory **Jobber** (OAuth + GraphQL, with refresh-token rotation) and **Google Calendar** (keeps cancelled events like Google does), served through a stubbed `fetch`

Covered: create, update, delete, reassign, two assignees, duplicates, unscheduled, Requests/Quotes/Assessments never synced, `JOB_CREATE`, all-day events, retries and throttling, token refresh and rotation, invalid refresh token, `APP_DISCONNECT`, no tokens in logs, HMAC, PKCE, the service-account JWT, every private-key paste format, and the admin pages.

Jobber → Google Calendar Sync — Source Code Appendix

The complete deployable source as of commit `7694c40`. The tests (`test/`, 65 tests) and docs are in the repo and in `jobber-gcal-sync-source.zip`; they're not reprinted here.

To rebuild from this appendix, recreate each file at the path shown, then run `npm install && npm test`.

Contents

- `wrangler.toml`
- `package.json`
- `tsconfig.json`
- `vitest.config.ts`
- `.dev.vars.example`
- `.gitignore`
- `migrations/0001_init.sql`
- `src/index.ts`
- `src/env.ts`
- `src/config.ts`
- `src/admin.ts`
- `src/adminUi.ts`
- `src/log.ts`
- `src/db/repo.ts`
- `src/google/auth.ts`
- `src/google/calendar.ts`
- `src/jobber/graphql.ts`
- `src/jobber/ids.ts`
- `src/jobber/oauth.ts`
- `src/jobber/tokens.ts`
- `src/jobber/webhook.ts`
- `src/sync/mapEvent.ts`
- `src/sync/processor.ts`
- `src/sync/validate.ts`
- `src/util/crypto.ts`
- `src/util/retry.ts`
- `graphql/introspect-webhook-topics.graphql`
- `graphql/visit-for-calendar.graphql`
- `scripts/sign-webhook.mjs`

`wrangler.toml`

```
name = "jobber-gcal-sync"
main = "src/index.ts"
compatibility_date = "2026-09-01"

# Keep variables added in the Cloudflare dashboard when GitHub pushes redeploy the Worker.
keep_vars = true

[observability]
enabled = true

# Non-secret configuration. Secrets are set with `wrangler secret put` (see README).
[vars]
# Latest active Jobber GraphQL version per the changelog (checked 2026-10-13).
JOBBER_API_VERSION = "2026-09-25"
# Must match the OAuth Callback URL configured on the Jobber app exactly.
JOBBER_REDIRECT_URI = "https://jobber-gcal-sync.joseramirez.workers.dev/oauth/jobber/callback"
# "service_account" (recommended) or "oauth"
GOOGLE_AUTH_MODE = "service_account"
# Used only for all-day visits (Google all-day events need a calendar date).
DEFAULT_TIMEZONE = "America/New_York"
# Webhook topics this worker will act on. Anything else is logged as "Skipped non-job event".
# Confirm the exact names in the Jobber Developer Center / GraphiQL (graphql/introspect-webhook-topics.graphql).
ALLOWED_TOPICS =
"VISIT_CREATE,VISIT_UPDATE,VISIT_COMPLETE,VISIT_DESTROY,JOB_CREATE,JOB_UPDATE,JOB_DESTROY,APP_DISCONNECT"
EVENT_TITLE_PREFIX = ""
# Not secret (public identifiers) — kept here so deploys never drop them.
JOBBER_CLIENT_ID = "8e03ed08-18b0-4477-bcb6-46b2ecaa1c13"
GOOGLE_SA_CLIENT_EMAIL = "jobber-sync@jobber-gcal-sync.iam.gserviceaccount.com"

[[d1_databases]]
binding = "DB"
database_name = "jobber_gcal_sync"
database_id = "f112efbe-2c51-4da7-911d-4c7adcbdfdff"
migrations_dir = "migrations"

[[queues.producers]]
binding = "SYNC_QUEUE"
queue = "jobber-visit-sync"

[[queues.consumers]]
queue = "jobber-visit-sync"
max_batch_size = 10
max_batch_timeout = 1
max_retries = 8
# One consumer at a time: serializes Jobber token refreshes and per-visit writes.
max_concurrency = 1
dead_letter_queue = "jobber-visit-sync-dlq"

# Daily housekeeping (prunes expired OAuth states and old sync_log rows).
[triggers]
crons = ["17 4 * * *"]
```

`package.json`

```
{
  "name": "jobber-gcal-sync",
  "version": "1.0.0",
  "private": true,
```

```
  "description": "Near real-time Jobber Job Visits -> Google Calendar sync on Cloudflare Workers",
  "type": "module",
  "scripts": {
    "dev": "wrangler dev",
    "deploy": "wrangler deploy",
    "test": "vitest run",
    "test:watch": "vitest",
    "typecheck": "tsc --noEmit",
    "db:migrate:local": "wrangler d1 migrations apply jobber_gcal_sync --local",
    "db:migrate:remote": "wrangler d1 migrations apply jobber_gcal_sync --remote",
    "sign": "node scripts/sign-webhook.mjs"
  },
  "devDependencies": {
    "@cloudflare/workers-types": "^5.20261013.1",
    "typescript": "^7.0.2",
    "vitest": "^5.0.4",
    "wrangler": "^4.150.0"
  }
}
```

`tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ES2022",
    "moduleResolution": "Bundler",
    "lib": ["ES2022"],
    "types": ["@cloudflare/workers-types"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noEmit": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src/**/*.ts"]
}
```

`vitest.config.ts`

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    environment: "node",
    include: ["test/**/*.test.ts"],
    testTimeout: 15000,
  },
});
```

`.dev.vars.example`

```
# Copy to .dev.vars for `wrangler dev` (never commit .dev.vars). In production use `wrangler secret put`.
JOBBER_CLIENT_ID=
JOBBER_CLIENT_SECRET=
```

```
# 32 random bytes, base64: openssl rand -base64 32
TOKEN_ENCRYPTION_KEY=
# Long random string for /admin/* and /oauth/jobber/start: openssl rand -hex 32
ADMIN_API_TOKEN=

# Google — service account mode (GOOGLE_AUTH_MODE=service_account)
GOOGLE_SA_CLIENT_EMAIL=
# Paste the private_key value from the JSON key file (keep the \n escapes)
GOOGLE_SA_PRIVATE_KEY=
# Optional, Workspace domain-wide delegation only
GOOGLE_SA_SUBJECT=

# Google — OAuth mode (GOOGLE_AUTH_MODE=oauth)
GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=
GOOGLE_OAUTH_REFRESH_TOKEN=

# Local override of the public callback (must also be added in the Jobber app if you test OAuth locally through
a tunnel)
# JOBBER_REDIRECT_URI=https://<your-tunnel>/oauth/jobber/callback
```

`.gitignore`

```
node_modules/
.wrangler/
.dev.vars
dist/
*.log
```

`migrations/0001_init.sql`

```
-- Jobber Visit -> Google Calendar sync schema (Cloudflare D1 / SQLite)

-- One row per connected Jobber account. Tokens are AES-GCM encrypted with TOKEN_ENCRYPTION_KEY.
CREATE TABLE IF NOT EXISTS jobber_accounts (
  account_id          TEXT PRIMARY KEY,           -- Jobber Account.id (EncodedId)
  account_name        TEXT,
  access_token_enc    TEXT NOT NULL,
  refresh_token_enc   TEXT NOT NULL,
  access_expires_at   INTEGER NOT NULL,           -- epoch ms
  refresh_lock_until  INTEGER,                    -- epoch ms; single-refresher lock (rotation safety)
  status              TEXT NOT NULL DEFAULT 'active', -- active | reauth_required | disconnected
  connected_at        INTEGER NOT NULL,
  updated_at          INTEGER NOT NULL
);

-- Short-lived OAuth state + PKCE verifier (10 minute TTL).
CREATE TABLE IF NOT EXISTS oauth_states (
  state         TEXT PRIMARY KEY,
  code_verifier TEXT NOT NULL,
  created_at    INTEGER NOT NULL
);

-- Team-member mapping: Jobber User.id -> Google Calendar ID. Platform IDs only (no names/emails as keys).
CREATE TABLE IF NOT EXISTS user_calendar_map (
  account_id          TEXT NOT NULL,
  jobber_user_id      TEXT NOT NULL,              -- Jobber User.id (EncodedId)
  google_calendar_id  TEXT NOT NULL,              -- e.g. estimator@company.com or xyz@group.calendar.google.com
  label               TEXT,                       -- human note only, never used for matching
```

```
  active              INTEGER NOT NULL DEFAULT 1,
  updated_at          INTEGER NOT NULL,
  PRIMARY KEY (account_id, jobber_user_id)
);

-- What we have written to Google. Written BEFORE each Google create so deletes always have the mapping,
-- even after the Visit is gone from Jobber.
CREATE TABLE IF NOT EXISTS visit_events (
  account_id          TEXT NOT NULL,
  visit_id            TEXT NOT NULL,
  jobber_user_id      TEXT NOT NULL,
  job_id              TEXT NOT NULL,
  google_calendar_id  TEXT NOT NULL,
  google_event_id     TEXT NOT NULL,              -- deterministic: "jv" + sha256(account|visit|user)[0..40]
  content_hash        TEXT,                       -- NULL until the Google write is confirmed
  last_synced_at      INTEGER NOT NULL,
  PRIMARY KEY (account_id, visit_id, jobber_user_id)
);
CREATE INDEX IF NOT EXISTS idx_visit_events_job ON visit_events (account_id, job_id);

-- Lightweight audit trail (mirrors the structured console logs; pruned after 30 days).
CREATE TABLE IF NOT EXISTS sync_log (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  at          INTEGER NOT NULL,
  topic       TEXT,
  account_id  TEXT,
  item_id     TEXT,
  job_id      TEXT,
  action      TEXT NOT NULL,                      -- created | updated | deleted | noop | skipped | error
  detail      TEXT
);
CREATE INDEX IF NOT EXISTS idx_sync_log_at ON sync_log (at);
```

src/index.ts

```
import type { Env, SyncMessage } from "./env";
import { classifyTopic } from "./config";
import { JOBBER_SIGNATURE_HEADER, parseJobberWebhook, verifyJobberSignature } from "./jobber/webhook";
import { handleJobberCallback, startJobberOAuth } from "./jobber/oauth";
import { handleAdmin, isAdmin } from "./admin";
import { handleAdminPage, handleJobberStartPage } from "./adminUi";
import { processMessage } from "./sync/processor";
import { PermanentError, RetryableError, queueBackoffSeconds } from "./util/retry";
import * as repo from "./db/repo";
import { log } from "./log";

/**
 * Jobber webhook -> verify HMAC -> enqueue IDs -> 200 (well under Jobber's 1s limit).
 * All Jobber/Google API work happens in the queue consumer.
 */
async function handleWebhook(env: Env, request: Request): Promise<Response> {
  const raw = await request.text();
  const ok = await verifyJobberSignature(raw, request.headers.get(JOBBER_SIGNATURE_HEADER),
env.JOBBER_CLIENT_SECRET);
  if (!ok) {
    log("warn", "Rejected webhook: invalid or missing signature", { bytes: raw.length });
    return new Response("invalid signature", { status: 401 });
  }

  const evt = parseJobberWebhook(raw);
  if (!evt) {
```

```
    log("warn", "Rejected webhook: unrecognized payload");
    return new Response("bad payload", { status: 400 });
  }

  const kind = classifyTopic(evt.topic, env);
  if (kind === "ignored") {
    log("info", "Skipped non-job event", { topic: evt.topic, itemId: evt.itemId, accountId: evt.accountId });
    return new Response("ignored", { status: 200 });
  }

  const msg: SyncMessage = { topic: evt.topic, accountId: evt.accountId, itemId: evt.itemId, occurredAt:
evt.occurredAt, receivedAt: Date.now() };
  await env.SYNC_QUEUE.send(msg);
  log("info", "webhook queued", { topic: evt.topic, itemId: evt.itemId, accountId: evt.accountId, kind });
  return new Response("ok", { status: 200 });
}

export default {
  async fetch(request: Request, env: Env, _ctx: ExecutionContext): Promise<Response> {
    const url = new URL(request.url);
    try {
      if (url.pathname === "/health") return new Response("ok");
      if (url.pathname === "/webhooks/jobber" && request.method === "POST") return await handleWebhook(env,
request);

      if (url.pathname === "/oauth/jobber/start" && (request.method === "GET" || request.method === "POST")) {
        if (request.method === "GET" && isAdmin(env, request)) return await startJobberOAuth(env);
        return await handleJobberStartPage(env, request, url);
      }
      if ((url.pathname === "/admin" || url.pathname === "/admin/") && (request.method === "GET" ||
request.method === "POST")) {
        return await handleAdminPage(env, request);
      }
      if (url.pathname === "/oauth/jobber/callback" && request.method === "GET") return await
handleJobberCallback(env, request);
      if (url.pathname.startsWith("/admin/")) return await handleAdmin(env, request, url);
      return new Response("not found", { status: 404 });
    } catch (err) {
      log("error", "request failed", { path: url.pathname, error: String(err) });
      return new Response("internal error", { status: 500 });
    }
  },

  async queue(batch: MessageBatch<SyncMessage>, env: Env): Promise<void> {
    // Collapse duplicates inside a batch (Jobber can fire the same topic ~1s apart for one action).
    const seen = new Set<string>();
    for (const message of batch.messages) {
      const m = message.body;
      const key = `${classifyTopic(m.topic, env)}|${m.accountId}|${m.itemId}`;
      if (seen.has(key)) {
        log("info", "duplicate in batch collapsed", { topic: m.topic, itemId: m.itemId });
        message.ack();
        continue;
      }
      seen.add(key);

      try {
        await processMessage(env, m);
        message.ack();
      } catch (err) {
        if (err instanceof PermanentError) {
          log("error", "sync failed permanently (not retried)", { topic: m.topic, itemId: m.itemId, accountId:
m.accountId, error: err.message });
```

```
          await repo.writeSyncLog(env.DB, { at: Date.now(), topic: m.topic, accountId: m.accountId, itemId:
m.itemId, action: "error", detail: err.message });
          message.ack();
        } else {
          const hint = err instanceof RetryableError ? err.retryAfterSeconds : undefined;
          const delaySeconds = queueBackoffSeconds(message.attempts, hint);
          log("warn", "sync failed; will retry", { topic: m.topic, itemId: m.itemId, attempts: message.attempts,
delaySeconds, error: String(err) });
          message.retry({ delaySeconds });
        }
      }
    }
  },

  async scheduled(_event: ScheduledController, env: Env): Promise<void> {
    await repo.prune(env.DB, Date.now());
  },
} satisfies ExportedHandler<Env, SyncMessage>;
```

src/env.ts

```
export interface Env {
  DB: D1Database;
  SYNC_QUEUE: Queue<SyncMessage>;

  // --- vars (wrangler.toml) ---
  JOBBER_API_VERSION: string;
  JOBBER_REDIRECT_URI: string;
  GOOGLE_AUTH_MODE: "service_account" | "oauth";
  DEFAULT_TIMEZONE: string;
  ALLOWED_TOPICS?: string;
  EVENT_TITLE_PREFIX?: string;

  // --- secrets (wrangler secret put) ---
  JOBBER_CLIENT_ID: string;
  JOBBER_CLIENT_SECRET: string; // also the webhook HMAC key
  TOKEN_ENCRYPTION_KEY: string; // base64, 32 random bytes
  ADMIN_API_TOKEN: string;

  // Google: service account mode
  GOOGLE_SA_CLIENT_EMAIL?: string;
  GOOGLE_SA_PRIVATE_KEY?: string; // PEM (PKCS#8), "\n" escapes allowed
  GOOGLE_SA_SUBJECT?: string; // optional: Workspace domain-wide delegation user to impersonate

  // Google: OAuth (shared admin account) mode
  GOOGLE_OAUTH_CLIENT_ID?: string;
  GOOGLE_OAUTH_CLIENT_SECRET?: string;
  GOOGLE_OAUTH_REFRESH_TOKEN?: string;
}

/** Message placed on the queue by the webhook endpoint. Contains IDs only. */
export interface SyncMessage {
  topic: string;
  accountId: string;
  itemId: string;
  occurredAt?: string;
  receivedAt: number;
}
```

src/config.ts

```
import type { Env } from "./env";

/**
 * Jobber topics follow OBJECT_ACTION (docs: "Setting up Webhooks"). The full list lives in the
 * Developer Center and in WebHookTopicEnum. Defaults below are the Visit/Job lifecycle topics;
 * override with ALLOWED_TOPICS if your Developer Center shows different names.
 */
export const DEFAULT_ALLOWED_TOPICS = [
  "VISIT_CREATE",
  "VISIT_UPDATE",
  "VISIT_COMPLETE",
  "VISIT_DESTROY",
  "JOB_CREATE",
  "JOB_UPDATE",
  "JOB_DESTROY",
  "APP_DISCONNECT",
];

export type TopicKind =
  | "visit_upsert"
  | "visit_delete"
  | "job_create"
  | "job_update"
  | "job_delete"
  | "app_disconnect"
  | "ignored";

const DELETE_ACTIONS = new Set(["DESTROY", "DELETE", "DESTROYED", "DELETED"]);

export function allowedTopics(env: Pick<Env, "ALLOWED_TOPICS">): Set<string> {
  const raw = env.ALLOWED_TOPICS?.trim();
  const list = raw ? raw.split(",").map((t) => t.trim().toUpperCase()).filter(Boolean) : DEFAULT_ALLOWED_TOPICS;
  return new Set(list);
}

/**
 * Classifies a topic. Only VISIT_*, JOB_* and APP_DISCONNECT can ever do work, and only if allow-listed.
 * REQUEST_*, QUOTE_*, ASSESSMENT_*, INVOICE_*, CLIENT_*, etc. are always "ignored".
 */
export function classifyTopic(topic: string, env: Pick<Env, "ALLOWED_TOPICS">): TopicKind {
  const t = topic.toUpperCase();
  if (!allowedTopics(env).has(t)) return "ignored";
  if (t === "APP_DISCONNECT") return "app_disconnect";
  const [object, ...rest] = t.split("_");
  const action = rest.join("_");
  if (object === "VISIT") return DELETE_ACTIONS.has(action) ? "visit_delete" : "visit_upsert";
  if (object === "JOB") {
    if (DELETE_ACTIONS.has(action)) return "job_delete";
    return action === "CREATE" ? "job_create" : "job_update";
  }
  return "ignored";
}
```

src/admin.ts

```
import type { Env } from "./env";
import * as repo from "./db/repo";
import { jobberGraphQL, USERS_QUERY } from "./jobber/graphql";
```

```
import { timingSafeEqual } from "./util/crypto";

export function isAdmin(env: Env, request: Request): boolean {
  const header = request.headers.get("authorization") ?? "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : "";
  return Boolean(env.ADMIN_API_TOKEN) && token.length > 0 && timingSafeEqual(token, env.ADMIN_API_TOKEN);
}

const json = (data: unknown, status = 200) =>
  new Response(JSON.stringify(data, null, 2), { status, headers: { "content-type": "application/json" } });

/**
 * Admin API (Bearer ADMIN_API_TOKEN):
 *   GET    /admin/accounts
 *   GET    /admin/jobber/users?accountId=…            Jobber users with IDs, to build the mapping
 *   GET    /admin/mappings[?accountId=…]
 *   PUT    /admin/mappings  {accountId, jobberUserId, googleCalendarId, label?, active?}
 *   DELETE /admin/mappings?accountId=…&jobberUserId=…
 *   GET    /admin/visits?accountId=…&visitId=…        stored visit -> event rows
 *   POST   /admin/resync {accountId, visitId}         re-run a visit through the queue
 *   GET    /admin/log?limit=50
 */
export async function handleAdmin(env: Env, request: Request, url: URL): Promise<Response> {
  if (!isAdmin(env, request)) return json({ error: "unauthorized" }, 401);
  const path = url.pathname;
  const q = url.searchParams;

  if (path === "/admin/accounts" && request.method === "GET") return json(await repo.listAccounts(env.DB));

  if (path === "/admin/jobber/users" && request.method === "GET") {
    const accountId = q.get("accountId");
    if (!accountId) return json({ error: "accountId required" }, 400);
    const data = await jobberGraphQL<{ users: { nodes: { id: string; name: { full: string } }[] } }>(env,
accountId, USERS_QUERY, { first: 100 });
    const mapped = await repo.listMappings(env.DB, accountId);
    return json(
      data.users.nodes.map((u) => ({
        jobberUserId: u.id,
        name: u.name?.full,
        googleCalendarId: mapped.find((m) => m.jobber_user_id === u.id)?.google_calendar_id ?? null,
      })),
    );
  }

  if (path === "/admin/mappings") {
    if (request.method === "GET") return json(await repo.listMappings(env.DB, q.get("accountId") ?? undefined));
    if (request.method === "PUT") {
      const b = (await request.json().catch(() => null)) as {
        accountId?: string;
        jobberUserId?: string;
        googleCalendarId?: string;
        label?: string;
        active?: boolean;
      } | null;
      if (!b?.accountId || !b.jobberUserId || !b.googleCalendarId) {
        return json({ error: "accountId, jobberUserId and googleCalendarId are required" }, 400);
      }
      await repo.upsertMapping(env.DB, { ...b, accountId: b.accountId, jobberUserId: b.jobberUserId,
googleCalendarId: b.googleCalendarId, now: Date.now() });
      return json({ ok: true });
    }
    if (request.method === "DELETE") {
      const accountId = q.get("accountId");
```

```
      const jobberUserId = q.get("jobberUserId");
      if (!accountId || !jobberUserId) return json({ error: "accountId and jobberUserId required" }, 400);
      await repo.deleteMapping(env.DB, accountId, jobberUserId);
      return json({ ok: true });
    }
  }

  if (path === "/admin/visits" && request.method === "GET") {
    const accountId = q.get("accountId");
    const visitId = q.get("visitId");
    if (!accountId || !visitId) return json({ error: "accountId and visitId required" }, 400);
    return json(await repo.listVisitEvents(env.DB, accountId, visitId));
  }

  if (path === "/admin/resync" && request.method === "POST") {
    const b = (await request.json().catch(() => null)) as { accountId?: string; visitId?: string } | null;
    if (!b?.accountId || !b.visitId) return json({ error: "accountId and visitId required" }, 400);
    await env.SYNC_QUEUE.send({ topic: "VISIT_UPDATE", accountId: b.accountId, itemId: b.visitId, receivedAt:
Date.now() });
    return json({ ok: true, queued: true });
  }

  if (path === "/admin/log" && request.method === "GET") {
    const limit = Math.min(500, Math.max(1, Number(q.get("limit") ?? 50) || 50));
    return json(await repo.recentSyncLog(env.DB, limit));
  }

  return json({ error: "not found" }, 404);
}
```

src/adminUi.ts

```
import type { Env } from "./env";
import * as repo from "./db/repo";
import { jobberGraphQL, USERS_QUERY } from "./jobber/graphql";
import { startJobberOAuth } from "./jobber/oauth";
import { timingSafeEqual } from "./util/crypto";

/**
 * Browser-friendly admin pages. The admin token is typed into a password box and POSTed,
 * so tokens containing #, &, + or $ never have to be URL-encoded.
 *   GET/POST /oauth/jobber/start   connect a Jobber account
 *   GET/POST /admin                map Jobber users -> Google Calendar IDs, see recent sync log
 */

function tokenOk(env: Env, key: string | null | undefined): boolean {
  return Boolean(env.ADMIN_API_TOKEN) && typeof key === "string" && key.length > 0 && timingSafeEqual(key,
env.ADMIN_API_TOKEN);
}

const esc = (s: unknown) =>
  String(s ?? "").replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'":
"&#39;" })[c]!);

function page(title: string, body: string, status = 200): Response {
  return new Response(
    `<!doctype html><html><head><meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-
scale=1">
<title>${esc(title)}</title><style>
:root{--bg:#f7f7f5;--fg:#1d1d1b;--muted:#6b6b66;--line:#e2e2dc;--accent:#2f6f4f;--err:#b3261e}
@media (prefers-color-scheme:dark){:root{--bg:#161615;--fg:#ededea;--muted:#a3a39c;--line:#2e2e2b;--
```

```
accent:#6fc196;--err:#f2b8b5}}
*{box-sizing:border-box}body{margin:0;background:var(--bg);color:var(--fg);font:15px/1.5 system-ui,-apple-
system,Segoe UI,sans-serif}
main{max-width:860px;margin:0 auto;padding:32px 16px}h1{font-size:22px;margin:0 0 4px}p.sub{color:var(--
muted);margin:0 0 24px}
input{font:inherit;padding:8px 10px;border:1px solid var(--line);border-
radius:8px;background:transparent;color:var(--fg);width:100%}
button{font:inherit;padding:8px 16px;border:0;border-radius:8px;background:var(--
accent);color:#fff;cursor:pointer}
table{width:100%;border-collapse:collapse;margin:8px 0 24px}th,td{text-align:left;padding:8px;border-bottom:1px
solid var(--line);vertical-align:middle}
th{font-size:12px;text-transform:uppercase;letter-spacing:.04em;color:var(--muted)}code{font-size:12px;word-
break:break-all;color:var(--muted)}
.err{color:var(--err)}.ok{color:var(--accent)}.row{display:flex;gap:8px;max-width:420px}.small{font-
size:13px;color:var(--muted)}
</style></head><body><main>${body}</main></body></html>`,
    { status, headers: { "content-type": "text/html; charset=utf-8", "cache-control": "no-store", "x-robots-
tag": "noindex" } },
  );
}

function loginForm(action: string, title: string, sub: string, error?: string): Response {
  return page(
    title,
    `<h1>${esc(title)}</h1><p class="sub">${esc(sub)}</p>
${error ? `<p class="err">${esc(error)}</p>` : ""}
<form method="post" action="${esc(action)}" class="row">
<input type="password" name="key" placeholder="ADMIN_API_TOKEN" autocomplete="current-password" required
autofocus>
<button type="submit">Continue</button></form>`,
    error ? 401 : 200,
  );
}

async function formData(request: Request): Promise<URLSearchParams> {
  return new URLSearchParams(await request.text());
}

/** /oauth/jobber/start */
export async function handleJobberStartPage(env: Env, request: Request, url: URL): Promise<Response> {
  if (request.method === "GET") {
    if (tokenOk(env, url.searchParams.get("key"))) return startJobberOAuth(env);
    return loginForm("/oauth/jobber/start", "Connect Jobber", "Enter the admin token to connect a Jobber
account.");
  }
  const f = await formData(request);
  if (!tokenOk(env, f.get("key"))) return loginForm("/oauth/jobber/start", "Connect Jobber", "Enter the admin
token to connect a Jobber account.", "That token didn't match ADMIN_API_TOKEN.");
  return startJobberOAuth(env);
}

/** /admin */
export async function handleAdminPage(env: Env, request: Request): Promise<Response> {
  const title = "Jobber → Google Calendar";
  if (request.method === "GET") return loginForm("/admin", title, "Enter the admin token to manage team
calendars.");

  const f = await formData(request);
  const key = f.get("key");
  if (!tokenOk(env, key)) return loginForm("/admin", title, "Enter the admin token to manage team calendars.",
"That token didn't match ADMIN_API_TOKEN.");

  let notice = "";
```

```
  if (f.get("action") === "save") {
    const accountId = f.get("accountId") ?? "";
    let saved = 0;
    for (const [name, value] of f) {
      if (!name.startsWith("cal:")) continue;
      const userId = name.slice(4);
      const label = f.get(`label:${userId}`) ?? null;
      const cal = value.trim();
      if (cal) {
        await repo.upsertMapping(env.DB, { accountId, jobberUserId: userId, googleCalendarId: cal, label, now:
Date.now() });
        saved++;
      } else {
        await repo.deleteMapping(env.DB, accountId, userId);
      }
    }
    notice = `<p class="ok">Saved ${saved} calendar mapping${saved === 1 ? "" : "s"}.</p>`;
  }

  const accounts = await repo.listAccounts(env.DB);
  if (accounts.length === 0) {
    return page(title, `<h1>${title}</h1><p class="sub">No Jobber account is connected yet.</p><p><a
href="/oauth/jobber/start">Connect Jobber →</a></p>`);
  }

  let sections = "";
  for (const a of accounts) {
    const mappings = await repo.listMappings(env.DB, a.account_id);
    let users: { id: string; name?: { full?: string } }[] = [];
    let usersError = "";
    if (a.status === "active") {
      try {
        users = (await jobberGraphQL<{ users: { nodes: { id: string; name: { full: string } }[] } }>(env,
a.account_id, USERS_QUERY, { first: 100 })).users.nodes;
      } catch (e) {
        usersError = String((e as Error).message ?? e);
      }
    }
    // Include mapped users that the API didn't return (e.g. deactivated).
    for (const m of mappings) if (!users.some((u) => u.id === m.jobber_user_id)) users.push({ id:
m.jobber_user_id, name: { full: m.label ?? "(not in Jobber list)" } });

    const rows = users
      .map((u) => {
        const m = mappings.find((x) => x.jobber_user_id === u.id);
        const name = u.name?.full ?? "";
        return `<tr><td>${esc(name)}<br><code>${esc(u.id)}</code><input type="hidden" name="label:${esc(u.id)}"
value="${esc(name)}"></td>
<td><input name="cal:${esc(u.id)}" value="${esc(m?.google_calendar_id ?? "")}"
placeholder="estimator@company.com"></td></tr>`;
      })
      .join("");

    sections += `<h2 style="font-size:17px;margin:24px 0 4px">${esc(a.account_name ?? "Jobber account")}</h2>
<p class="small">Status: <b>${esc(a.status)}</b>${a.status !== "active" ? ` — <a
href="/oauth/jobber/start">reconnect</a>` : ""}<br>Account ID: <code>${esc(a.account_id)}</code></p>
${usersError ? `<p class="err">Couldn't load Jobber users: ${esc(usersError)}</p>` : ""}
<form method="post" action="/admin"><input type="hidden" name="key" value="${esc(key)}"><input type="hidden"
name="action" value="save"><input type="hidden" name="accountId" value="${esc(a.account_id)}">
<table><thead><tr><th>Jobber team member</th><th>Google Calendar ID (leave blank to stop syncing)</th></tr>
</thead><tbody>${rows || `<tr><td colspan="2" class="small">No users found.</td></tr>`}</tbody></table>
<button type="submit">Save mappings</button></form>`;
  }
```

```
  const log = (await repo.recentSyncLog(env.DB, 25)) as Record<string, unknown>[];
  const logRows = log
    .map(
      (r) =>
        `<tr><td class="small">${esc(new Date(Number(r.at)).toLocaleString("en-US"))}</td><td>${esc(r.action)}
</td><td class="small">${esc(r.topic)}</td><td class="small">${esc(String(r.detail ?? "").slice(0, 160))}</td>
</tr>`,
    )
    .join("");

  return page(
    title,
    `<h1>${title}</h1><p class="sub">Each team member's Jobber visits are written to the Google Calendar listed
here. Share that calendar with <code>${esc(env.GOOGLE_SA_CLIENT_EMAIL ?? "the service account")}</code> ("Make
changes to events") first.</p>
${notice}${sections}
<h2 style="font-size:17px;margin:32px 0 4px">Recent activity</h2>
<table><thead><tr><th>When</th><th>Action</th><th>Topic</th><th>Detail</th></tr></thead><tbody>${logRows ||
`<tr><td colspan="4" class="small">Nothing yet.</td></tr>`}</tbody></table>`,
  );
}
```

src/log.ts

```
/** Structured JSON logging. Keys that look like credentials are always redacted. */
const SENSITIVE_KEY = /token|secret|authorization|password|private_?key|assertion|code_verifier|^code$/i;

function redact(value: unknown, depth = 0): unknown {
  if (depth > 6 || value === null || typeof value !== "object") return value;
  if (Array.isArray(value)) return value.map((v) => redact(v, depth + 1));
  const out: Record<string, unknown> = {};
  for (const [k, v] of Object.entries(value as Record<string, unknown>)) {
    out[k] = SENSITIVE_KEY.test(k) ? "[REDACTED]" : redact(v, depth + 1);
  }
  return out;
}

export type LogLevel = "debug" | "info" | "warn" | "error";

export function log(level: LogLevel, msg: string, fields: Record<string, unknown> = {}): void {
  const line = JSON.stringify({ level, msg, ts: new Date().toISOString(), ...(redact(fields) as object) });
  if (level === "error") console.error(line);
  else if (level === "warn") console.warn(line);
  else console.log(line);
}

/** Truncate API bodies before logging so a large or odd response never floods logs. */
export function snippet(text: string, max = 500): string {
  return text.length > max ? `${text.slice(0, max)}…` : text;
}
```

src/db/repo.ts

```
/** All D1 access lives here. */

export interface AccountRow {
  account_id: string;
  account_name: string | null;
```

```
  access_token_enc: string;
  refresh_token_enc: string;
  access_expires_at: number;
  refresh_lock_until: number | null;
  status: "active" | "reauth_required" | "disconnected";
  connected_at: number;
  updated_at: number;
}

export interface VisitEventRow {
  account_id: string;
  visit_id: string;
  jobber_user_id: string;
  job_id: string;
  google_calendar_id: string;
  google_event_id: string;
  content_hash: string | null;
  last_synced_at: number;
}

export interface MappingRow {
  account_id: string;
  jobber_user_id: string;
  google_calendar_id: string;
  label: string | null;
  active: number;
  updated_at: number;
}

// ---- accounts / tokens ------------------------------------------------------------

export async function getAccount(db: D1Database, accountId: string): Promise<AccountRow | null> {
  return db.prepare("SELECT * FROM jobber_accounts WHERE account_id = ?").bind(accountId).first<AccountRow>();
}

export async function listAccounts(db: D1Database): Promise<Omit<AccountRow, "access_token_enc" |
"refresh_token_enc">[]> {
  const { results } = await db
    .prepare(
      "SELECT account_id, account_name, access_expires_at, refresh_lock_until, status, connected_at, updated_at
FROM jobber_accounts ORDER BY connected_at",
    )
    .all<Omit<AccountRow, "access_token_enc" | "refresh_token_enc">>();
  return results;
}

export async function upsertAccountTokens(
  db: D1Database,
  a: { accountId: string; accountName: string | null; accessEnc: string; refreshEnc: string; expiresAt: number;
now: number },
): Promise<void> {
  await db
    .prepare(
      `INSERT INTO jobber_accounts (account_id, account_name, access_token_enc, refresh_token_enc,
access_expires_at, refresh_lock_until, status, connected_at, updated_at)
       VALUES (?, ?, ?, ?, ?, NULL, 'active', ?, ?)
       ON CONFLICT(account_id) DO UPDATE SET
         account_name = COALESCE(excluded.account_name, jobber_accounts.account_name),
         access_token_enc = excluded.access_token_enc,
         refresh_token_enc = excluded.refresh_token_enc,
         access_expires_at = excluded.access_expires_at,
         refresh_lock_until = NULL,
         status = 'active',
```

```
          updated_at = excluded.updated_at`,
    )
    .bind(a.accountId, a.accountName, a.accessEnc, a.refreshEnc, a.expiresAt, a.now, a.now)
    .run();
}

/** Atomically claims the refresh lock. Returns true if this caller may refresh. */
export async function claimRefreshLock(db: D1Database, accountId: string, now: number, ttlMs: number):
Promise<boolean> {
  const res = await db
    .prepare(
      "UPDATE jobber_accounts SET refresh_lock_until = ? WHERE account_id = ? AND (refresh_lock_until IS NULL OR
refresh_lock_until < ?)",
    )
    .bind(now + ttlMs, accountId, now)
    .run();
  return (res.meta?.changes ?? 0) > 0;
}

export async function releaseRefreshLock(db: D1Database, accountId: string): Promise<void> {
  await db.prepare("UPDATE jobber_accounts SET refresh_lock_until = NULL WHERE account_id =
?").bind(accountId).run();
}

export async function setAccountStatus(db: D1Database, accountId: string, status: AccountRow["status"], now:
number) {
  await db
    .prepare("UPDATE jobber_accounts SET status = ?, refresh_lock_until = NULL, updated_at = ? WHERE account_id
= ?")
    .bind(status, now, accountId)
    .run();
}

// ---- oauth state ------------------------------------------------------------------------

export async function saveOAuthState(db: D1Database, state: string, verifier: string, now: number) {
  await db.prepare("INSERT INTO oauth_states (state, code_verifier, created_at) VALUES (?, ?, ?)").bind(state,
verifier, now).run();
}

/** Single-use: reads and deletes. */
export async function consumeOAuthState(db: D1Database, state: string): Promise<{ code_verifier: string;
created_at: number } | null> {
  const row = await db
    .prepare("SELECT code_verifier, created_at FROM oauth_states WHERE state = ?")
    .bind(state)
    .first<{ code_verifier: string; created_at: number }>();
  if (row) await db.prepare("DELETE FROM oauth_states WHERE state = ?").bind(state).run();
  return row;
}

// ---- team-member mapping ----------------------------------------------------------------

export async function getCalendarsForUsers(db: D1Database, accountId: string, userIds: string[]):
Promise<Map<string, string>> {
  const map = new Map<string, string>();
  if (userIds.length === 0) return map;
  const placeholders = userIds.map(() => "?").join(",");
  const { results } = await db
    .prepare(
      `SELECT jobber_user_id, google_calendar_id FROM user_calendar_map WHERE account_id = ? AND active = 1 AND
jobber_user_id IN (${placeholders})`,
    )
```

```
    .bind(accountId, ...userIds)
    .all<{ jobber_user_id: string; google_calendar_id: string }>();
  for (const r of results) map.set(r.jobber_user_id, r.google_calendar_id);
  return map;
}

export async function listMappings(db: D1Database, accountId?: string): Promise<MappingRow[]> {
  const stmt = accountId
    ? db.prepare("SELECT * FROM user_calendar_map WHERE account_id = ? ORDER BY label").bind(accountId)
    : db.prepare("SELECT * FROM user_calendar_map ORDER BY account_id, label");
  return (await stmt.all<MappingRow>()).results;
}

export async function upsertMapping(
  db: D1Database,
  m: { accountId: string; jobberUserId: string; googleCalendarId: string; label?: string | null; active?:
boolean; now: number },
) {
  await db
    .prepare(
      `INSERT INTO user_calendar_map (account_id, jobber_user_id, google_calendar_id, label, active, updated_at)
       VALUES (?, ?, ?, ?, ?, ?)
       ON CONFLICT(account_id, jobber_user_id) DO UPDATE SET
         google_calendar_id = excluded.google_calendar_id, label = excluded.label,
         active = excluded.active, updated_at = excluded.updated_at`,
    )
    .bind(m.accountId, m.jobberUserId, m.googleCalendarId, m.label ?? null, m.active === false ? 0 : 1, m.now)
    .run();
}

export async function deleteMapping(db: D1Database, accountId: string, jobberUserId: string) {
  await db.prepare("DELETE FROM user_calendar_map WHERE account_id = ? AND jobber_user_id = ?").bind(accountId,
jobberUserId).run();
}

// ---- visit -> event mapping ------------------------------------------------------------

export async function listVisitEvents(db: D1Database, accountId: string, visitId: string):
Promise<VisitEventRow[]> {
  return (
    await db
      .prepare("SELECT * FROM visit_events WHERE account_id = ? AND visit_id = ?")
      .bind(accountId, visitId)
      .all<VisitEventRow>()
  ).results;
}

export async function listVisitIdsForJob(db: D1Database, accountId: string, jobId: string): Promise<string[]> {
  const { results } = await db
    .prepare("SELECT DISTINCT visit_id FROM visit_events WHERE account_id = ? AND job_id = ?")
    .bind(accountId, jobId)
    .all<{ visit_id: string }>();
  return results.map((r) => r.visit_id);
}

/** Written before the Google call (content_hash NULL) and again after it succeeds (with the hash). */
export async function upsertVisitEvent(db: D1Database, r: VisitEventRow) {
  await db
    .prepare(
      `INSERT INTO visit_events (account_id, visit_id, jobber_user_id, job_id, google_calendar_id,
google_event_id, content_hash, last_synced_at)
       VALUES (?, ?, ?, ?, ?, ?, ?, ?)
       ON CONFLICT(account_id, visit_id, jobber_user_id) DO UPDATE SET
```

```
          job_id = excluded.job_id, google_calendar_id = excluded.google_calendar_id,
          google_event_id = excluded.google_event_id, content_hash = excluded.content_hash,
          last_synced_at = excluded.last_synced_at`,
    )
    .bind(r.account_id, r.visit_id, r.jobber_user_id, r.job_id, r.google_calendar_id, r.google_event_id,
r.content_hash, r.last_synced_at)
    .run();
}

export async function deleteVisitEvent(db: D1Database, accountId: string, visitId: string, jobberUserId: string)
{
  await db
    .prepare("DELETE FROM visit_events WHERE account_id = ? AND visit_id = ? AND jobber_user_id = ?")
    .bind(accountId, visitId, jobberUserId)
    .run();
}

// ---- audit log ---------------------------------------------------------------------------

export async function writeSyncLog(
  db: D1Database,
  e: { at: number; topic?: string; accountId?: string; itemId?: string; jobId?: string; action: string; detail?:
unknown },
) {
  try {
    await db
      .prepare("INSERT INTO sync_log (at, topic, account_id, item_id, job_id, action, detail) VALUES (?, ?, ?,
?, ?, ?, ?)")
      .bind(
        e.at,
        e.topic ?? null,
        e.accountId ?? null,
        e.itemId ?? null,
        e.jobId ?? null,
        e.action,
        e.detail === undefined ? null : typeof e.detail === "string" ? e.detail : JSON.stringify(e.detail),
      )
      .run();
  } catch {
    // Audit logging must never break a sync.
  }
}

export async function recentSyncLog(db: D1Database, limit: number) {
  return (await db.prepare("SELECT * FROM sync_log ORDER BY id DESC LIMIT ?").bind(limit).all()).results;
}

export async function prune(db: D1Database, now: number) {
  await db.prepare("DELETE FROM oauth_states WHERE created_at < ?").bind(now - 60 * 60 * 1000).run();
  await db.prepare("DELETE FROM sync_log WHERE at < ?").bind(now - 30 * 24 * 60 * 60 * 1000).run();
}
```

`src/google/auth.ts`

```
import type { Env } from "../env";
import { b64decode, b64url } from "../util/crypto";
import { fetchWithRetry, PermanentError, RetryableError } from "../util/retry";
import { log, snippet } from "../log";

export const GOOGLE_TOKEN_URL = "https://oauth2.googleapis.com/token";
/** Minimum scope: read/write events only (no calendar list/ACL/settings access). */
```

```ts
export const GOOGLE_CALENDAR_SCOPE = "https://www.googleapis.com/auth/calendar.events";

let cached: { token: string; expiresAt: number } | null = null;

export function resetGoogleTokenCache(): void {
  cached = null;
}

/** Access token for the Calendar API, cached in isolate memory until ~1 minute before expiry. */
export async function getGoogleAccessToken(env: Env): Promise<string> {
  if (cached && cached.expiresAt - 60_000 > Date.now()) return cached.token;
  const res =
    env.GOOGLE_AUTH_MODE === "oauth" ? await refreshOAuthToken(env) : await serviceAccountToken(env);
  const text = await res.text();
  if (res.status === 400 || res.status === 401) {
    log("error", "Google token request rejected", { status: res.status, response: snippet(text, 200) });
    throw new PermanentError("Google credentials rejected; check GOOGLE_* secrets");
  }
  if (!res.ok) throw new RetryableError(`Google token endpoint HTTP ${res.status}`);
  const json = JSON.parse(text) as { access_token: string; expires_in?: number };
  cached = { token: json.access_token, expiresAt: Date.now() + (json.expires_in ?? 3600) * 1000 };
  return json.access_token;
}

async function refreshOAuthToken(env: Env): Promise<Response> {
  if (!env.GOOGLE_OAUTH_CLIENT_ID || !env.GOOGLE_OAUTH_CLIENT_SECRET || !env.GOOGLE_OAUTH_REFRESH_TOKEN) {
    throw new PermanentError("GOOGLE_AUTH_MODE=oauth requires GOOGLE_OAUTH_CLIENT_ID,
GOOGLE_OAUTH_CLIENT_SECRET, GOOGLE_OAUTH_REFRESH_TOKEN");
  }
  return fetchWithRetry(
    GOOGLE_TOKEN_URL,
    {
      method: "POST",
      headers: { "content-type": "application/x-www-form-urlencoded" },
      body: new URLSearchParams({
        grant_type: "refresh_token",
        client_id: env.GOOGLE_OAUTH_CLIENT_ID,
        client_secret: env.GOOGLE_OAUTH_CLIENT_SECRET,
        refresh_token: env.GOOGLE_OAUTH_REFRESH_TOKEN,
      }).toString(),
    },
    { label: "google.oauth.token" },
  );
}

async function serviceAccountToken(env: Env): Promise<Response> {
  if (!env.GOOGLE_SA_CLIENT_EMAIL || !env.GOOGLE_SA_PRIVATE_KEY) {
    throw new PermanentError("GOOGLE_AUTH_MODE=service_account requires GOOGLE_SA_CLIENT_EMAIL and
GOOGLE_SA_PRIVATE_KEY");
  }
  const assertion = await buildServiceAccountJwt(env.GOOGLE_SA_CLIENT_EMAIL, env.GOOGLE_SA_PRIVATE_KEY,
env.GOOGLE_SA_SUBJECT);
  return fetchWithRetry(
    GOOGLE_TOKEN_URL,
    {
      method: "POST",
      headers: { "content-type": "application/x-www-form-urlencoded" },
      body: new URLSearchParams({ grant_type: "urn:ietf:params:oauth:grant-type:jwt-bearer", assertion
}).toString(),
    },
    { label: "google.sa.token" },
  );
}
```

```
/** RS256-signed JWT bearer assertion (RFC 7523) for a Google service account. */
export async function buildServiceAccountJwt(clientEmail: string, privateKeyPem: string, subject?: string,
nowSec = Math.floor(Date.now() / 1000)): Promise<string> {
  const header = { alg: "RS256", typ: "JWT" };
  const claims: Record<string, unknown> = {
    iss: clientEmail,
    scope: GOOGLE_CALENDAR_SCOPE,
    aud: GOOGLE_TOKEN_URL,
    iat: nowSec,
    exp: nowSec + 3600,
  };
  if (subject) claims.sub = subject;
  const unsigned = `${b64url(JSON.stringify(header))}.${b64url(JSON.stringify(claims))}`;
  const key = await importPkcs8(privateKeyPem);
  const sig = await crypto.subtle.sign("RSASSA-PKCS1-v1_5", key, new TextEncoder().encode(unsigned));
  return `${unsigned}.${b64url(sig)}`;
}

/**
 * Accepts the private key however it was pasted into the dashboard:
 *  - the raw PEM, with real newlines or literal "\n" (or "\\n") escapes
 *  - the JSON string value including surrounding quotes and/or a trailing comma
 *  - the entire service-account JSON file
 * Returns only the base64 body between the BEGIN/END markers.
 */
export function normalizePrivateKey(input: string): string {
  let s = input.trim();
  if (s.startsWith("{")) {
    try {
      const parsed = JSON.parse(s) as { private_key?: string };
      if (parsed.private_key) s = parsed.private_key;
    } catch {
      /* not valid JSON; fall through */
    }
  }
  s = s.replace(/\\\\n/g, "\n").replace(/\\n/g, "\n").replace(/\\r/g, "");
  const m = /-----BEGIN (?:RSA )?PRIVATE KEY-----([\s\S]*?)-----END (?:RSA )?PRIVATE KEY-----/.exec(s);
  const body = m ? m[1]! : s;
  return body.replace(/[^A-Za-z0-9+/=]/g, "");
}

/** Non-secret shape info, safe to log, to diagnose a bad paste. */
export function describePrivateKey(input: string): Record<string, unknown> {
  const body = normalizePrivateKey(input);
  return {
    rawLength: input.length,
    hasBeginMarker: /BEGIN (RSA )?PRIVATE KEY/.test(input),
    hasEndMarker: /END (RSA )?PRIVATE KEY/.test(input),
    startsWithBrace: input.trim().startsWith("{"),
    startsWithQuote: input.trim().startsWith('"'),
    base64Length: body.length,
  };
}

async function importPkcs8(pem: string): Promise<CryptoKey> {
  const body = normalizePrivateKey(pem);
  try {
    return await crypto.subtle.importKey("pkcs8", b64decode(body), { name: "RSASSA-PKCS1-v1_5", hash: "SHA-256"
}, false, ["sign"]);
  } catch (err) {
    log("error", "GOOGLE_SA_PRIVATE_KEY could not be read — re-paste the private_key value from the JSON key
file", describePrivateKey(pem));
```

```
    // Retryable on purpose: visits queued while the secret is wrong sync once it is fixed.
    throw new RetryableError(`GOOGLE_SA_PRIVATE_KEY is not a valid private key (${String(err)})`, 120);
  }
}
```

src/google/calendar.ts

```
import type { Env } from "../env";
import { getGoogleAccessToken, resetGoogleTokenCache } from "./auth";
import { fetchWithRetry, PermanentError, RetryableError } from "../util/retry";
import { log, snippet } from "../log";

export const GCAL_BASE = "https://www.googleapis.com/calendar/v3";

export type GEventTime = { dateTime: string; timeZone?: string } | { date: string };

export interface GEvent {
  id: string;
  summary: string;
  location?: string;
  description?: string;
  start: GEventTime;
  end: GEventTime;
  status: "confirmed";
  transparency: "opaque"; // shows as busy -> blocks availability in Routemize
  extendedProperties: { private: Record<string, string> };
  source?: { title: string; url: string };
  reminders: { useDefault: boolean };
}

function eventUrl(calendarId: string, eventId?: string): string {
  const base = `${GCAL_BASE}/calendars/${encodeURIComponent(calendarId)}/events`;
  return eventId ? `${base}/${encodeURIComponent(eventId)}` : base;
}

async function gcal(env: Env, method: string, url: string, body?: unknown): Promise<{ status: number; text:
string }> {
  for (let attempt = 0; attempt < 2; attempt++) {
    const token = await getGoogleAccessToken(env);
    const res = await fetchWithRetry(
      `${url}${url.includes("?") ? "&" : "?"}sendUpdates=none`,
      {
        method,
        headers: { authorization: `Bearer ${token}`, "content-type": "application/json" },
        body: body === undefined ? undefined : JSON.stringify(body),
      },
      { label: `google.calendar.${method.toLowerCase()}` },
    );
    const text = await res.text();
    if (res.status === 401 && attempt === 0) {
      resetGoogleTokenCache();
      continue;
    }
    if (res.status === 403 && /rateLimitExceeded|userRateLimitExceeded|quotaExceeded/i.test(text)) {
      throw new RetryableError("Google Calendar rate limited", 30);
    }
    if (res.status === 429 || res.status >= 500) throw new RetryableError(`Google Calendar HTTP ${res.status}`);
    return { status: res.status, text };
  }
  throw new PermanentError("Google Calendar rejected credentials after refresh");
}
```

```
/**
 * Idempotent create-or-update with a deterministic event ID:
 *   PUT (update)  -> 200 updated (also revives an event previously deleted/cancelled with this ID)
 *   404           -> POST (insert) with our ID -> created
 *   409 on insert -> someone created it in between -> PUT again
 */
export async function upsertCalendarEvent(env: Env, calendarId: string, event: GEvent): Promise<"created" |
"updated"> {
  const put = await gcal(env, "PUT", eventUrl(calendarId, event.id), event);
  if (put.status === 200) return "updated";
  if (put.status !== 404) throw calendarError("update", calendarId, put);

  const ins = await gcal(env, "POST", eventUrl(calendarId), event);
  if (ins.status === 200) return "created";
  if (ins.status === 409) {
    const retry = await gcal(env, "PUT", eventUrl(calendarId, event.id), event);
    if (retry.status === 200) return "updated";
    throw calendarError("update", calendarId, retry);
  }
  throw calendarError("insert", calendarId, ins);
}

/** Delete; 404/410 count as already gone. */
export async function deleteCalendarEvent(env: Env, calendarId: string, eventId: string): Promise<"deleted" |
"already_gone"> {
  const res = await gcal(env, "DELETE", eventUrl(calendarId, eventId));
  if (res.status === 204 || res.status === 200) return "deleted";
  if (res.status === 404 || res.status === 410) return "already_gone";
  throw calendarError("delete", calendarId, res);
}

function calendarError(op: string, calendarId: string, r: { status: number; text: string }): Error {
  log("error", "Google Calendar API error", { op, calendarId, status: r.status, response: snippet(r.text, 300)
});
  // 403/404 here mean the calendar isn't shared with our identity (or doesn't exist): a config problem.
  return new PermanentError(`Google Calendar ${op} failed on ${calendarId}: HTTP ${r.status}`);
}
```

src/jobber/graphql.ts

```
import type { Env } from "../env";
import { getJobberAccessToken } from "./tokens";
import { fetchWithRetry, PermanentError, RetryableError } from "../util/retry";
import { log, snippet } from "../log";

export const JOBBER_GRAPHQL_URL = "https://api.getjobber.com/api/graphql";

/**
 * Visit + parent Job details. Field names are from the published schema in the Developer Center
 * (type Visit / Job / Property / Client / User). Connection is paginated (`first`) to keep query cost low.
 * Keep in sync with graphql/visit-for-calendar.graphql.
 */
export const VISIT_QUERY = /* GraphQL */ `
  query VisitForCalendar($id: EncodedId!) {
    visit(id: $id) {
      __typename
      id
      title
      startAt
      endAt
```

```
      allDay
      instructions
      visitStatus
      isComplete
      job {
        __typename
        id
        jobNumber
        title
        jobberWebUri
      }
      client {
        id
        name
      }
      property {
        id
        address {
          street1
          street2
          city
          province
          postalCode
          country
        }
      }
      assignedUsers(first: 25) {
        nodes {
          id
          name {
            full
          }
        }
      }
    }
  }
`;

/** Visit IDs of one Job (used for JOB_CREATE). Paginated; capped by MAX_JOB_VISITS in the processor. */
export const JOB_VISITS_QUERY = /* GraphQL */ `
  query JobVisits($id: EncodedId!, $first: Int!, $after: String) {
    job(id: $id) {
      id
      visits(first: $first, after: $after) {
        nodes {
          id
        }
        pageInfo {
          hasNextPage
          endCursor
        }
      }
    }
  }
`;

export const USERS_QUERY = /* GraphQL */ `
  query JobberUsers($first: Int!) {
    users(first: $first) {
      nodes {
        id
        name {
          full
```

```
        }
      }
    }
  }
`;

export const ACCOUNT_QUERY = /* GraphQL */ `
  query GetAccount {
    account {
      id
      name
    }
  }
`;

export interface JobberUserRef {
  id: string;
  name?: { full?: string | null } | null;
}

export interface JobberVisit {
  __typename?: string;
  id: string;
  title: string | null;
  startAt: string | null;
  endAt: string | null;
  allDay: boolean;
  instructions: string | null;
  visitStatus: string;
  isComplete: boolean;
  job: { __typename?: string; id: string; jobNumber: number | null; title: string | null; jobberWebUri: string |
null } | null;
  client: { id: string; name: string | null } | null;
  property: {
    id: string;
    address: {
      street1?: string | null;
      street2?: string | null;
      city?: string | null;
      province?: string | null;
      postalCode?: string | null;
      country?: string | null;
    } | null;
  } | null;
  assignedUsers: { nodes: JobberUserRef[] } | null;
}

interface GraphQLError {
  message: string;
  extensions?: { code?: string };
}

const NOT_FOUND = /not\s*found|could not find|does not exist|couldn't find/i;

/**
 * POSTs a GraphQL operation for one Jobber account.
 * - 401 -> forces one token refresh and retries.
 * - HTTP 429 / 5xx -> retried inline, then RetryableError (queue retries later).
 * - errors[].extensions.code === "THROTTLED" -> RetryableError with a wait derived from the cost bucket.
 * - "not found" errors with a null result -> returns data so callers can treat the record as gone.
 */
export async function jobberGraphQL<T>(env: Env, accountId: string, query: string, variables: Record<string,
unknown> = {}, accessToken?: string): Promise<T> {
```

```
  let forced = false;
  let token = accessToken ?? (await getJobberAccessToken(env, accountId));

  for (;;) {
    const res = await fetchWithRetry(
      JOBBER_GRAPHQL_URL,
      {
        method: "POST",
        headers: {
          authorization: `Bearer ${token}`,
          "content-type": "application/json",
          "X-JOBBER-GRAPHQL-VERSION": env.JOBBER_API_VERSION,
        },
        body: JSON.stringify({ query, variables }),
      },
      { label: "jobber.graphql" },
    );

    if (res.status === 401 && !forced && !accessToken) {
      forced = true;
      await res.body?.cancel();
      token = await getJobberAccessToken(env, accountId, { forceRefresh: true });
      continue;
    }
    const text = await res.text();
    if (res.status === 429 || res.status >= 500) throw new RetryableError(`Jobber GraphQL HTTP ${res.status}`);
    if (!res.ok) throw new PermanentError(`Jobber GraphQL HTTP ${res.status}: ${snippet(text, 200)}`);

    const body = JSON.parse(text) as {
      data?: T;
      errors?: GraphQLError[];
      extensions?: {
        versioning?: { version?: string; warning?: string };
        cost?: { requestedQueryCost?: number; throttleStatus?: { currentlyAvailable?: number; restoreRate?:
number } };
      };
    };

    const versionWarning = body.extensions?.versioning?.warning;
    if (versionWarning) log("warn", "Jobber API version warning", { warning: versionWarning });

    if (body.errors?.length) {
      if (body.errors.some((e) => e.extensions?.code === "THROTTLED")) {
        const c = body.extensions?.cost;
        const deficit = (c?.requestedQueryCost ?? 0) - (c?.throttleStatus?.currentlyAvailable ?? 0);
        const wait = Math.ceil(Math.max(deficit, 0) / Math.max(c?.throttleStatus?.restoreRate ?? 500, 1)) + 1;
        throw new RetryableError("Jobber GraphQL throttled", wait);
      }
      if (body.data && body.errors.every((e) => NOT_FOUND.test(e.message))) return body.data;
      log("error", "Jobber GraphQL errors", { errors: body.errors.map((e) => e.message) });
      throw new PermanentError(`Jobber GraphQL error: ${body.errors.map((e) => e.message).join("; ")}`);
    }
    if (!body.data) throw new RetryableError("Jobber GraphQL returned no data");
    return body.data;
  }
}
```

src/jobber/ids.ts

```
/**
 * Jobber IDs are EncodedId: base64 of a global ID, e.g. "Z2lkOi8vSm9iYmVyL0NsaWVudC8xMTkxOTUzNDA" ->
```

```
 * "gid://Jobber/Client/119195340" (docs: "Object IDs (EncodedId)"). We use this only as an extra guard:
 * if an ID clearly decodes to a non-Visit type, it is never synced.
 */
export function decodeEncodedId(id: string): { type: string; numericId: string } | null {
  try {
    const decoded = atob(id);
    const m = /^gid:\/\/Jobber\/([A-Za-z]+)\/(\d+)$/.exec(decoded);
    return m ? { type: m[1]!, numericId: m[2]! } : null;
  } catch {
    return null;
  }
}

/** True when the ID decodes to a type other than `expected`. Undecodable IDs return false (no evidence). */
export function isDefinitelyNot(id: string, expected: string): boolean {
  const d = decodeEncodedId(id);
  return d !== null && d.type !== expected;
}
```

src/jobber/oauth.ts

```
import type { Env } from "../env";
import * as repo from "../db/repo";
import { pkceChallenge, randomToken } from "../util/crypto";
import { JOBBER_AUTHORIZE_URL, postTokenEndpoint, storeTokens, type JobberTokenResponse } from "./tokens";
import { ACCOUNT_QUERY, jobberGraphQL } from "./graphql";
import { log, snippet } from "../log";

const STATE_TTL_MS = 10 * 60 * 1000; // authorization codes also expire after 10 minutes

/** GET /oauth/jobber/start — redirects a Jobber admin to the consent screen (PKCE S256 + state). */
export async function startJobberOAuth(env: Env): Promise<Response> {
  const state = randomToken(24);
  const verifier = randomToken(48); // 64 unreserved chars (43–128 required)
  await repo.saveOAuthState(env.DB, state, verifier, Date.now());

  const url = new URL(JOBBER_AUTHORIZE_URL);
  url.searchParams.set("response_type", "code");
  url.searchParams.set("client_id", env.JOBBER_CLIENT_ID);
  url.searchParams.set("redirect_uri", env.JOBBER_REDIRECT_URI);
  url.searchParams.set("state", state);
  url.searchParams.set("code_challenge", await pkceChallenge(verifier));
  url.searchParams.set("code_challenge_method", "S256");
  return Response.redirect(url.toString(), 302);
}

/** GET /oauth/jobber/callback?code=…&state=… — exchanges the code, stores tokens keyed by Jobber account.id. */
export async function handleJobberCallback(env: Env, request: Request): Promise<Response> {
  const url = new URL(request.url);
  const code = url.searchParams.get("code");
  const state = url.searchParams.get("state");
  if (!code || !state) return html(400, "Missing code or state. Start again from /oauth/jobber/start.");

  const saved = await repo.consumeOAuthState(env.DB, state);
  if (!saved || Date.now() - saved.created_at > STATE_TTL_MS) {
    return html(400, "This authorization link is invalid or expired. Start again from /oauth/jobber/start.");
  }

  const res = await postTokenEndpoint(env, {
    grant_type: "authorization_code",
    code,
```

```
    redirect_uri: env.JOBBER_REDIRECT_URI,
    code_verifier: saved.code_verifier,
  });
  const text = await res.text();
  if (!res.ok) {
    log("error", "Jobber code exchange failed", { status: res.status, response: snippet(text, 200) });
    return html(502, "Jobber rejected the authorization code. Start again from /oauth/jobber/start.");
  }
  const tokens = JSON.parse(text) as JobberTokenResponse;

  // Recommended by Jobber: identify the account the tokens belong to.
  const data = await jobberGraphQL<{ account: { id: string; name: string } }>(env, "", ACCOUNT_QUERY, {},
tokens.access_token);
  await storeTokens(env, data.account.id, data.account.name, tokens);
  log("info", "Jobber account connected", { accountId: data.account.id, accountName: data.account.name });

  return html(
    200,
    `Connected Jobber account <b>${escapeHtml(data.account.name)}</b>.<br>Account ID:
<code>${escapeHtml(data.account.id)}</code><br>Next: <a href="/admin">map team members to their Google
Calendars</a>.`,
  );
}

function html(status: number, body: string): Response {
  return new Response(`<!doctype html><meta charset="utf-8"><title>Jobber connection</title><body style="font-
family:system-ui;padding:2rem;max-width:40rem">${body}</body>`, {
    status,
    headers: { "content-type": "text/html; charset=utf-8" },
  });
}

function escapeHtml(s: string): string {
  return s.replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" })
[c]!);
}
```

src/jobber/tokens.ts

```
import type { Env } from "../env";
import * as repo from "../db/repo";
import { decryptString, encryptString } from "../util/crypto";
import { fetchWithRetry, PermanentError, RetryableError } from "../util/retry";
import { log, snippet } from "../log";

export const JOBBER_TOKEN_URL = "https://api.getjobber.com/api/oauth/token";
export const JOBBER_AUTHORIZE_URL = "https://api.getjobber.com/api/oauth/authorize";

/** Refresh this long before expiry (access tokens live 60 minutes). */
const REFRESH_SKEW_MS = 5 * 60 * 1000;
const LOCK_TTL_MS = 30 * 1000;

export interface JobberTokenResponse {
  access_token: string;
  refresh_token: string;
  token_type?: string;
  expires_in?: number;
  warning?: string;
}

/** Access tokens are JWTs with `exp`; fall back to expires_in, then to 60 minutes. */
```

```
export function accessTokenExpiry(t: JobberTokenResponse, now: number): number {
  try {
    const payload = JSON.parse(atob(t.access_token.split(".")[1]!.replace(/-/g, "+").replace(/_/g, "/")));
    if (typeof payload.exp === "number") return payload.exp * 1000;
  } catch {
    /* not a decodable JWT */
  }
  return now + (t.expires_in ?? 3600) * 1000;
}

export async function storeTokens(env: Env, accountId: string, accountName: string | null, t:
JobberTokenResponse): Promise<void> {
  const now = Date.now();
  await repo.upsertAccountTokens(env.DB, {
    accountId,
    accountName,
    accessEnc: await encryptString(env.TOKEN_ENCRYPTION_KEY, t.access_token),
    refreshEnc: await encryptString(env.TOKEN_ENCRYPTION_KEY, t.refresh_token),
    expiresAt: accessTokenExpiry(t, now),
    now,
  });
}

export async function postTokenEndpoint(env: Env, params: Record<string, string>): Promise<Response> {
  return fetchWithRetry(
    JOBBER_TOKEN_URL,
    {
      method: "POST",
      headers: { "content-type": "application/x-www-form-urlencoded", accept: "application/json" },
      body: new URLSearchParams({ client_id: env.JOBBER_CLIENT_ID, client_secret: env.JOBBER_CLIENT_SECRET,
...params }).toString(),
    },
    { label: "jobber.oauth.token" },
  );
}

/**
 * Returns a valid access token for the account, refreshing when needed.
 *
 * Refresh Token Rotation is on by default and the previous refresh token is invalidated immediately,
 * so exactly one process may refresh at a time: we take a short D1 lock, refresh, and persist BOTH
 * new tokens before using the new access token (per Jobber's rotation guidance). Other callers wait
 * and re-read the stored tokens instead of refreshing themselves.
 */
export async function getJobberAccessToken(env: Env, accountId: string, opts: { forceRefresh?: boolean } = {}):
Promise<string> {
  for (let attempt = 0; attempt < 6; attempt++) {
    const row = await repo.getAccount(env.DB, accountId);
    if (!row) throw new PermanentError(`Jobber account ${accountId} is not connected`);
    if (row.status !== "active") throw new PermanentError(`Jobber account ${accountId} status is ${row.status};
re-authorize via /oauth/jobber/start`);

    const now = Date.now();
    const fresh = row.access_expires_at - REFRESH_SKEW_MS > now;
    if (fresh && !(opts.forceRefresh && attempt === 0)) {
      return decryptString(env.TOKEN_ENCRYPTION_KEY, row.access_token_enc);
    }

    if (await repo.claimRefreshLock(env.DB, accountId, now, LOCK_TTL_MS)) {
      try {
        return await refreshWithLock(env, row);
      } finally {
        await repo.releaseRefreshLock(env.DB, accountId);
```

```
      }
    }
    // Someone else is refreshing; wait then read what they stored.
    await new Promise((r) => setTimeout(r, 500 * (attempt + 1)));
    opts = {};
  }
  throw new RetryableError(`Timed out waiting for Jobber token refresh for ${accountId}`);
}

async function refreshWithLock(env: Env, row: repo.AccountRow): Promise<string> {
  const refreshToken = await decryptString(env.TOKEN_ENCRYPTION_KEY, row.refresh_token_enc);
  const res = await postTokenEndpoint(env, { grant_type: "refresh_token", refresh_token: refreshToken });
  const text = await res.text();

  if (res.status === 400 || res.status === 401) {
    // invalid_grant / "The provided refresh token is not valid." -> user must re-authorize.
    await repo.setAccountStatus(env.DB, row.account_id, "reauth_required", Date.now());
    log("error", "Jobber refresh token rejected; account needs re-authorization", {
      accountId: row.account_id,
      status: res.status,
      response: snippet(text, 200),
    });
    throw new PermanentError(`Jobber refresh token rejected for ${row.account_id}`);
  }
  if (!res.ok) throw new RetryableError(`Jobber token refresh failed: HTTP ${res.status}`);

  const tokens = JSON.parse(text) as JobberTokenResponse;
  if (tokens.warning) log("warn", "Jobber token endpoint warning", { accountId: row.account_id, warning:
tokens.warning });
  if (!tokens.access_token || !tokens.refresh_token) throw new RetryableError("Jobber token refresh returned no
tokens");

  // Persist both tokens before any further API call.
  await storeTokens(env, row.account_id, null, tokens);
  log("info", "Jobber access token refreshed", { accountId: row.account_id });
  return tokens.access_token;
}
```

src/jobber/webhook.ts

```
import { hmacSha256Base64, timingSafeEqual } from "../util/crypto";

export const JOBBER_SIGNATURE_HEADER = "X-Jobber-Hmac-SHA256";

export interface JobberWebhookEvent {
  topic: string;
  appId?: string;
  accountId: string;
  itemId: string;
  occurredAt?: string;
}

/**
 * Jobber signs every webhook: header X-Jobber-Hmac-SHA256 = base64(HMAC-SHA256(client_secret, raw body)).
 * Must be computed over the exact raw bytes received, before JSON parsing.
 */
export async function verifyJobberSignature(rawBody: string, header: string | null, clientSecret: string):
Promise<boolean> {
  if (!header || !clientSecret) return false;
  const expected = await hmacSha256Base64(clientSecret, rawBody);
  return timingSafeEqual(expected, header.trim());
```

```
}

/**
 * Payload: { data: { webHookEvent: { topic, appId, accountId, itemId, occurredAt } } }.
 * Apps created before 2023-12-08 receive the misspelled `occuredAt`; both are accepted.
 */
export function parseJobberWebhook(rawBody: string): JobberWebhookEvent | null {
  let parsed: unknown;
  try {
    parsed = JSON.parse(rawBody);
  } catch {
    return null;
  }
  const e = (parsed as { data?: { webHookEvent?: Record<string, unknown> } })?.data?.webHookEvent;
  if (!e || typeof e.topic !== "string" || typeof e.accountId !== "string" || typeof e.itemId !== "string")
return null;
  const occurred = (e.occurredAt ?? e.occuredAt) as string | undefined;
  return {
    topic: e.topic,
    appId: typeof e.appId === "string" ? e.appId : undefined,
    accountId: e.accountId,
    itemId: e.itemId,
    occurredAt: typeof occurred === "string" ? occurred : undefined,
  };
}
```

src/sync/mapEvent.ts

```
import type { Env } from "../env";
import type { GEvent, GEventTime } from "../google/calendar";
import type { ValidJobVisit } from "./validate";
import { sha256Hex } from "../util/crypto";

/**
 * Deterministic Google event ID per (account, visit, assigned user).
 * Google allows base32hex chars [a-v0-9], length 5–1024; hex digits are a subset, so "jv" + 40 hex is valid.
 * Same webhook processed twice -> same ID -> update, never a duplicate.
 */
export async function deterministicEventId(accountId: string, visitId: string, jobberUserId: string):
Promise<string> {
  return `jv${(await sha256Hex(`${accountId}|${visitId}|${jobberUserId}`)).slice(0, 40)}`;
}

export function formatAddress(a: NonNullable<NonNullable<ValidJobVisit["property"]>["address"]> | null |
undefined): string {
  if (!a) return "";
  const cityLine = [a.city, [a.province, a.postalCode].filter(Boolean).join(" ")].filter(Boolean).join(", ");
  return [a.street1, a.street2, cityLine, a.country].map((s) => s?.trim()).filter(Boolean).join(", ");
}

function localDate(iso: string, timeZone: string): string {
  // en-CA formats as YYYY-MM-DD
  return new Intl.DateTimeFormat("en-CA", { timeZone, year: "numeric", month: "2-digit", day: "2-digit"
}).format(new Date(iso));
}

function addDays(ymd: string, days: number): string {
  const d = new Date(`${ymd}T00:00:00Z`);
  d.setUTCDate(d.getUTCDate() + days);
  return d.toISOString().slice(0, 10);
}
```

```
/** Google all-day events use dates with an exclusive end. */
function eventTimes(visit: ValidJobVisit, tz: string): { start: GEventTime; end: GEventTime } {
  if (!visit.allDay) return { start: { dateTime: visit.startAt }, end: { dateTime: visit.endAt } };
  const startDate = localDate(visit.startAt, tz);
  const lastDay = localDate(new Date(Date.parse(visit.endAt) - 1000).toISOString(), tz);
  const endExclusive = addDays(lastDay < startDate ? startDate : lastDay, 1);
  return { start: { date: startDate }, end: { date: endExclusive } };
}

export function buildEventBody(env: Pick<Env, "DEFAULT_TIMEZONE" | "EVENT_TITLE_PREFIX">, accountId: string,
visit: ValidJobVisit, jobberUserId: string, eventId: string): GEvent {
  const job = visit.job;
  const title =
    visit.title?.trim() || job.title?.trim() || (job.jobNumber != null ? `Job #${job.jobNumber}` : "Jobber
visit");
  const prefix = env.EVENT_TITLE_PREFIX?.trim();
  const address = formatAddress(visit.property?.address);

  const description = [
    visit.client?.name ? `Client: ${visit.client.name}` : null,
    job.jobNumber != null ? `Job #${job.jobNumber}${job.title ? ` — ${job.title}` : ""}` : null,
    visit.instructions?.trim() ? `\nInstructions:\n${visit.instructions.trim()}` : null,
    job.jobberWebUri ? `\nOpen in Jobber: ${job.jobberWebUri}` : null,
    "\n(Synced from Jobber — edit the visit in Jobber, not here.)",
  ]
    .filter(Boolean)
    .join("\n");

  const event: GEvent = {
    id: eventId,
    summary: prefix ? `${prefix} ${title}` : title,
    location: address || undefined,
    description,
    ...eventTimes(visit, env.DEFAULT_TIMEZONE || "UTC"),
    status: "confirmed",
    transparency: "opaque",
    reminders: { useDefault: false },
    extendedProperties: {
      private: {
        source: "jobber-visit-sync",
        jobberAccountId: accountId,
        jobberVisitId: visit.id,
        jobberJobId: job.id,
        jobberUserId,
        jobberVisitStatus: visit.visitStatus ?? "",
      },
    },
  };
  if (job.jobberWebUri) event.source = { title: "Jobber", url: job.jobberWebUri };
  return event;
}

/** Hash of what we'd write; identical hash on the same calendar = nothing to do (duplicate webhook). */
export async function contentHash(calendarId: string, event: GEvent): Promise<string> {
  return sha256Hex(`${calendarId}\n${JSON.stringify(event)}`);
}
```

src/sync/processor.ts

```
import type { Env, SyncMessage } from "../env";
import * as repo from "../db/repo";
import { classifyTopic } from "../config";
import { jobberGraphQL, JOB_VISITS_QUERY, VISIT_QUERY, type JobberVisit } from "../jobber/graphql";
import { isDefinitelyNot } from "../jobber/ids";
import { deleteCalendarEvent, upsertCalendarEvent } from "../google/calendar";
import { buildEventBody, contentHash, deterministicEventId } from "./mapEvent";
import { validateJobVisit } from "./validate";
import { log } from "../log";

export type SyncAction = "created" | "updated" | "deleted" | "noop" | "skipped";

interface Ctx {
  env: Env;
  topic: string;
  accountId: string;
}

async function record(ctx: Ctx, fields: { action: SyncAction | "error"; itemId?: string; jobId?: string;
userId?: string; calendarId?: string; reason?: string; detail?: unknown }) {
  const level = fields.action === "error" ? "error" : "info";
  log(level, `sync ${fields.action}`, {
    topic: ctx.topic,
    accountId: ctx.accountId,
    visitId: fields.itemId,
    jobId: fields.jobId,
    assignedUser: fields.userId,
    calendarId: fields.calendarId,
    reason: fields.reason,
    detail: fields.detail,
  });
  await repo.writeSyncLog(ctx.env.DB, {
    at: Date.now(),
    topic: ctx.topic,
    accountId: ctx.accountId,
    itemId: fields.itemId,
    jobId: fields.jobId,
    action: fields.action,
    detail: { reason: fields.reason, userId: fields.userId, calendarId: fields.calendarId, ...(fields.detail as
object | undefined) },
  });
}

/** Entry point for one queue message. Throws RetryableError / PermanentError for the queue handler. */
export async function processMessage(env: Env, msg: SyncMessage): Promise<void> {
  const ctx: Ctx = { env, topic: msg.topic, accountId: msg.accountId };
  const kind = classifyTopic(msg.topic, env);

  switch (kind) {
    case "ignored":
      await record(ctx, { action: "skipped", itemId: msg.itemId, reason: "Skipped non-job event" });
      return;

    case "app_disconnect":
      await repo.setAccountStatus(env.DB, msg.accountId, "disconnected", Date.now());
      await record(ctx, { action: "skipped", itemId: msg.itemId, reason: "Jobber app disconnected; tokens
disabled" });
      return;

    case "visit_upsert":
```

```
      if (isDefinitelyNot(msg.itemId, "Visit")) {
        await record(ctx, { action: "skipped", itemId: msg.itemId, reason: "Skipped non-job event" });
        return;
      }
      await syncVisit(ctx, msg.itemId);
      return;

    case "visit_delete":
      if (isDefinitelyNot(msg.itemId, "Visit")) {
        await record(ctx, { action: "skipped", itemId: msg.itemId, reason: "Skipped non-job event" });
        return;
      }
      await removeVisit(ctx, msg.itemId, "visit deleted in Jobber");
      return;

    case "job_create": {
      // Safety net: Jobber also sends VISIT_CREATE per visit, but sync every visit of a new job in case one is
missed.
      const visitIds = await listJobVisitIds(env, msg.accountId, msg.itemId);
      for (const id of visitIds) await syncVisit(ctx, id);
      if (visitIds.length === 0) await record(ctx, { action: "noop", jobId: msg.itemId, reason: "new job has no
visits yet" });
      return;
    }

    case "job_update": {
      // Job title/property/assignment edits may not fire per-visit webhooks; re-sync visits we track.
      const visitIds = await repo.listVisitIdsForJob(env.DB, msg.accountId, msg.itemId);
      for (const id of visitIds) await syncVisit(ctx, id);
      if (visitIds.length === 0) await record(ctx, { action: "noop", jobId: msg.itemId, reason: "no tracked
visits for job" });
      return;
    }

    case "job_delete": {
      const visitIds = await repo.listVisitIdsForJob(env.DB, msg.accountId, msg.itemId);
      for (const id of visitIds) await removeVisit(ctx, id, "parent job deleted in Jobber");
      return;
    }
  }
}

export const MAX_JOB_VISITS = 100;

async function listJobVisitIds(env: Env, accountId: string, jobId: string): Promise<string[]> {
  const ids: string[] = [];
  let after: string | null = null;
  while (ids.length < MAX_JOB_VISITS) {
    const data: { job: { visits: { nodes: { id: string }[]; pageInfo: { hasNextPage: boolean; endCursor: string
| null } } } | null } =
      await jobberGraphQL(env, accountId, JOB_VISITS_QUERY, { id: jobId, first: 25, after });
    if (!data.job) break;
    ids.push(...data.job.visits.nodes.map((n) => n.id));
    if (!data.job.visits.pageInfo.hasNextPage || !data.job.visits.pageInfo.endCursor) break;
    after = data.job.visits.pageInfo.endCursor;
  }
  return ids.slice(0, MAX_JOB_VISITS);
}

/** Fetch current state from Jobber and make Google match it. Order-independent and idempotent. */
export async function syncVisit(ctx: Ctx, visitId: string): Promise<void> {
  const { env, accountId } = ctx;
  const data = await jobberGraphQL<{ visit: JobberVisit | null }>(env, accountId, VISIT_QUERY, { id: visitId });
```

```
  const result = validateJobVisit(data.visit);
  const existing = await repo.listVisitEvents(env.DB, accountId, visitId);

  if (!result.ok) {
    await record(ctx, { action: "skipped", itemId: visitId, jobId: data.visit?.job?.id, reason: result.message
});
    // A visit that stopped qualifying (unscheduled, unassigned, gone) must not keep blocking calendars.
    if (existing.length > 0 && result.reason !== "non_job_object") {
      await removeVisit(ctx, visitId, `no longer a scheduled job visit (${result.reason})`, existing);
    }
    return;
  }

  const visit = result.visit;
  const calendars = await repo.getCalendarsForUsers(env.DB, accountId, visit.users.map((u) => u.id));
  const desired = new Map<string, string>(); // jobberUserId -> calendarId
  for (const u of visit.users) {
    const cal = calendars.get(u.id);
    if (cal) desired.set(u.id, cal);
    else await record(ctx, { action: "skipped", itemId: visitId, jobId: visit.job.id, userId: u.id, reason: `No
calendar mapping for Jobber user ${u.name?.full ?? u.id}` });
  }

  // 1) Reassignment / calendar change: remove events from calendars that should no longer have them.
  for (const row of existing) {
    const target = desired.get(row.jobber_user_id);
    if (target === row.google_calendar_id) continue;
    const r = await deleteCalendarEvent(env, row.google_calendar_id, row.google_event_id);
    await repo.deleteVisitEvent(env.DB, accountId, visitId, row.jobber_user_id);
    await record(ctx, { action: "deleted", itemId: visitId, jobId: row.job_id, userId: row.jobber_user_id,
calendarId: row.google_calendar_id, reason: target ? "calendar mapping changed" : "user unassigned", detail: {
google: r } });
  }

  // 2) Create/update one event per mapped assigned user.
  for (const [userId, calendarId] of desired) {
    const eventId = await deterministicEventId(accountId, visitId, userId);
    const event = buildEventBody(env, accountId, visit, userId, eventId);
    const hash = await contentHash(calendarId, event);
    const prev = existing.find((r) => r.jobber_user_id === userId && r.google_calendar_id === calendarId);
    if (prev?.content_hash === hash) {
      await record(ctx, { action: "noop", itemId: visitId, jobId: visit.job.id, userId, calendarId, reason:
"unchanged (duplicate webhook)" });
      continue;
    }

    const row: repo.VisitEventRow = {
      account_id: accountId,
      visit_id: visitId,
      jobber_user_id: userId,
      job_id: visit.job.id,
      google_calendar_id: calendarId,
      google_event_id: eventId,
      content_hash: null,
      last_synced_at: Date.now(),
    };
    // Persist the mapping BEFORE writing to Google, so a later delete can always find the event.
    await repo.upsertVisitEvent(env.DB, row);
    const outcome = await upsertCalendarEvent(env, calendarId, event);
    await repo.upsertVisitEvent(env.DB, { ...row, content_hash: hash, last_synced_at: Date.now() });
    await record(ctx, { action: outcome, itemId: visitId, jobId: visit.job.id, userId, calendarId, detail: {
eventId, startAt: visit.startAt, endAt: visit.endAt } });
  }
```

```
}

/** Delete every event we created for a visit, using stored mappings (the Visit may no longer exist in Jobber).
*/
export async function removeVisit(ctx: Ctx, visitId: string, reason: string, rows?: repo.VisitEventRow[]):
Promise<void> {
  const { env, accountId } = ctx;
  const existing = rows ?? (await repo.listVisitEvents(env.DB, accountId, visitId));
  if (existing.length === 0) {
    await record(ctx, { action: "noop", itemId: visitId, reason: `${reason}; nothing to delete` });
    return;
  }
  for (const row of existing) {
    const r = await deleteCalendarEvent(env, row.google_calendar_id, row.google_event_id);
    await repo.deleteVisitEvent(env.DB, accountId, visitId, row.jobber_user_id);
    await record(ctx, { action: "deleted", itemId: visitId, jobId: row.job_id, userId: row.jobber_user_id,
calendarId: row.google_calendar_id, reason, detail: { google: r } });
  }
}
```

src/sync/validate.ts

```
import type { JobberUserRef, JobberVisit } from "../jobber/graphql";
import { isDefinitelyNot } from "../jobber/ids";

export type SkipReason = "not_found" | "non_job_object" | "unscheduled" | "no_assigned_users";

export interface ValidJobVisit extends JobberVisit {
  startAt: string;
  endAt: string;
  job: NonNullable<JobberVisit["job"]>;
  users: JobberUserRef[];
}

export type ValidationResult = { ok: true; visit: ValidJobVisit } | { ok: false; reason: SkipReason; message:
string };

/** Log messages required by the spec, keyed by reason. */
export const SKIP_MESSAGES: Record<SkipReason, string> = {
  not_found: "Skipped: visit not found (deleted or not a Visit)",
  non_job_object: "Skipped non-job event",
  unscheduled: "Skipped unscheduled job visit",
  no_assigned_users: "Skipped job visit with no assigned users",
};

/**
 * Gate run before EVERY Google Calendar write: "Is this a scheduled Visit attached to a Job?"
 *  1. it is a Visit (GraphQL __typename, and the ID doesn't decode to another type)
 *  2. it belongs to a Job (Visit.job is `Job!` in the schema; still checked defensively)
 *  3. startAt and endAt are both present (unscheduled visits have both null)
 *  4. at least one assigned user
 * Requests, Quotes and Assessments are separate types (Assessment.request: Request!) and can never pass.
 */
export function validateJobVisit(visit: JobberVisit | null | undefined): ValidationResult {
  const skip = (reason: SkipReason): ValidationResult => ({ ok: false, reason, message: SKIP_MESSAGES[reason]
});

  if (!visit) return skip("not_found");
  if ((visit.__typename && visit.__typename !== "Visit") || isDefinitelyNot(visit.id, "Visit")) return
skip("non_job_object");
  if (!visit.job?.id || (visit.job.__typename && visit.job.__typename !== "Job") ||
```

```
isDefinitelyNot(visit.job.id, "Job")) {
    return skip("non_job_object");
  }
  if (!visit.startAt || !visit.endAt) return skip("unscheduled");

  const users = (visit.assignedUsers?.nodes ?? []).filter((u): u is JobberUserRef => Boolean(u?.id));
  if (users.length === 0) return skip("no_assigned_users");

  return { ok: true, visit: { ...visit, startAt: visit.startAt, endAt: visit.endAt, job: visit.job, users } };
}
```

src/util/crypto.ts

```
const encoder = new TextEncoder();
const decoder = new TextDecoder();

export function b64encode(input: ArrayBuffer | Uint8Array): string {
  const bytes = input instanceof Uint8Array ? input : new Uint8Array(input);
  let s = "";
  for (let i = 0; i < bytes.length; i++) s += String.fromCharCode(bytes[i]!);
  return btoa(s);
}

export function b64decode(s: string): Uint8Array {
  const bin = atob(s);
  const out = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++) out[i] = bin.charCodeAt(i);
  return out;
}

export function b64url(input: ArrayBuffer | Uint8Array | string): string {
  const bytes = typeof input === "string" ? encoder.encode(input) : input;
  return b64encode(bytes).replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
}

export function randomToken(byteLength = 32): string {
  return b64url(crypto.getRandomValues(new Uint8Array(byteLength)));
}

export async function sha256Hex(data: string): Promise<string> {
  const digest = await crypto.subtle.digest("SHA-256", encoder.encode(data));
  return [...new Uint8Array(digest)].map((b) => b.toString(16).padStart(2, "0")).join("");
}

/** PKCE S256 code_challenge = base64url(sha256(verifier)) without padding. */
export async function pkceChallenge(verifier: string): Promise<string> {
  return b64url(await crypto.subtle.digest("SHA-256", encoder.encode(verifier)));
}

/** base64(HMAC-SHA256(secret, rawBody)) — the format Jobber sends in X-Jobber-Hmac-SHA256. */
export async function hmacSha256Base64(secret: string, data: string): Promise<string> {
  const key = await crypto.subtle.importKey("raw", encoder.encode(secret), { name: "HMAC", hash: "SHA-256" },
false, [
    "sign",
  ]);
  return b64encode(await crypto.subtle.sign("HMAC", key, encoder.encode(data)));
}

/** Constant-time string comparison (does not short-circuit on the first differing byte). */
export function timingSafeEqual(a: string, b: string): boolean {
  const ab = encoder.encode(a);
```

```
  const bb = encoder.encode(b);
  const len = Math.max(ab.length, bb.length);
  let diff = ab.length ^ bb.length;
  for (let i = 0; i < len; i++) diff |= (ab[i] ?? 0) ^ (bb[i] ?? 0);
  return diff === 0;
}

// ---- AES-GCM for tokens at rest -----------------------------------------------------------

const keyCache = new Map<string, Promise<CryptoKey>>();

/**
 * TOKEN_ENCRYPTION_KEY: ideally 32 random bytes, base64. Any other string (a passphrase) is accepted and
 * stretched to a 256-bit key with SHA-256, so a hand-typed secret still works.
 */
async function rawKeyBytes(secret: string): Promise<Uint8Array> {
  try {
    const raw = b64decode(secret.trim());
    if (raw.length === 32) return raw;
  } catch {
    /* not base64 — treat as passphrase */
  }
  return new Uint8Array(await crypto.subtle.digest("SHA-256", encoder.encode(secret)));
}

function aesKey(secret: string): Promise<CryptoKey> {
  let k = keyCache.get(secret);
  if (!k) {
    if (!secret) throw new Error("TOKEN_ENCRYPTION_KEY is not set");
    k = rawKeyBytes(secret).then((raw) => crypto.subtle.importKey("raw", raw, "AES-GCM", false, ["encrypt",
"decrypt"]));
    keyCache.set(secret, k);
  }
  return k;
}

export async function encryptString(base64Key: string, plaintext: string): Promise<string> {
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const ct = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, await aesKey(base64Key),
encoder.encode(plaintext));
  return `v1:${b64encode(iv)}:${b64encode(ct)}`;
}

export async function decryptString(base64Key: string, payload: string): Promise<string> {
  const [version, ivB64, ctB64] = payload.split(":");
  if (version !== "v1" || !ivB64 || !ctB64) throw new Error("Unrecognized encrypted token format");
  const pt = await crypto.subtle.decrypt(
    { name: "AES-GCM", iv: b64decode(ivB64) },
    await aesKey(base64Key),
    b64decode(ctB64),
  );
  return decoder.decode(pt);
}
```

`src/util/retry.ts`

```
import { log } from "../log";

/** Temporary failure: the queue message should be retried later (with backoff). */
export class RetryableError extends Error {
  constructor(
```

```
    message: string,
    public retryAfterSeconds?: number,
  ) {
    super(message);
    this.name = "RetryableError";
  }
}

/** Failure that will not fix itself (bad config, revoked auth, calendar not shared). Ack + log. */
export class PermanentError extends Error {
  constructor(message: string) {
    super(message);
    this.name = "PermanentError";
  }
}

/** Tunable so tests don't sleep. */
export const retryConfig = { baseDelayMs: 400, maxInlineWaitMs: 5000 };

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

function isRetryableStatus(status: number): boolean {
  return status === 429 || (status >= 500 && status <= 599);
}

function parseRetryAfter(header: string | null): number | undefined {
  if (!header) return undefined;
  const secs = Number(header);
  if (Number.isFinite(secs)) return Math.max(0, secs);
  const date = Date.parse(header);
  return Number.isFinite(date) ? Math.max(0, Math.ceil((date - Date.now()) / 1000)) : undefined;
}

/**
 * Short in-request retries (jittered exponential backoff) for network errors, 429 and 5xx.
 * If the server asks for a longer wait than we're willing to spend inline, throws RetryableError
 * so the queue retries later instead. All callers send idempotent requests, so replays are safe.
 * Returns the final Response (which may still be an error status) for the caller to interpret.
 */
export async function fetchWithRetry(
  url: string,
  init: RequestInit,
  opts: { attempts?: number; label: string },
): Promise<Response> {
  const attempts = opts.attempts ?? 3;
  let lastError: unknown;
  for (let attempt = 1; attempt <= attempts; attempt++) {
    let res: Response;
    try {
      res = await fetch(url, init);
    } catch (err) {
      lastError = err;
      log("warn", "network error", { label: opts.label, attempt, error: String(err) });
      if (attempt < attempts) await sleep(backoff(attempt));
      continue;
    }
    if (!isRetryableStatus(res.status) || attempt === attempts) return res;

    const retryAfter = parseRetryAfter(res.headers.get("retry-after"));
    if (retryAfter !== undefined && retryAfter * 1000 > retryConfig.maxInlineWaitMs) {
      throw new RetryableError(`${opts.label}: HTTP ${res.status}, retry after ${retryAfter}s`, retryAfter);
    }
    log("warn", "retrying request", { label: opts.label, attempt, status: res.status });
```

```
    await res.body?.cancel();
    await sleep(retryAfter !== undefined ? retryAfter * 1000 : backoff(attempt));
  }
  throw new RetryableError(`${opts.label}: network failure after ${attempts} attempts: ${String(lastError)}`);
}

function backoff(attempt: number): number {
  const base = retryConfig.baseDelayMs * 2 ** (attempt - 1);
  return base / 2 + Math.random() * (base / 2);
}

/** Queue-level backoff: 15s, 30s, 60s … capped at 15 minutes. */
export function queueBackoffSeconds(attempts: number, hint?: number): number {
  const exp = Math.min(900, 15 * 2 ** Math.max(0, attempts - 1));
  return Math.max(exp, Math.min(900, hint ?? 0));
}
```

graphql/introspect-webhook-topics.graphql

```
# Run once in Jobber's GraphiQL to see the exact webhook topic names your API version supports.
# The Worker defaults to VISIT_CREATE, VISIT_UPDATE, VISIT_COMPLETE, VISIT_DESTROY, JOB_CREATE, JOB_UPDATE,
JOB_DESTROY, APP_DISCONNECT.
# If the names differ (e.g. VISIT_DELETE), update ALLOWED_TOPICS in wrangler.toml — no code change needed.
query WebhookTopics {
  __type(name: "WebHookTopicEnum") {
    enumValues {
      name
      description
    }
  }
}

# Also worth confirming once (field names used by the sync):
query VisitFields {
  visit: __type(name: "Visit") { fields { name type { name kind ofType { name } } } }
  address: __type(name: "PropertyAddress") { fields { name } }
  name: __type(name: "Name") { fields { name } }
}
```

graphql/visit-for-calendar.graphql

```
# The exact query the Worker runs for every VISIT_* webhook (mirrors VISIT_QUERY in src/jobber/graphql.ts).
# Paste into Jobber's GraphiQL with variables {"id": "<itemId from a webhook>"} to confirm it against your API
version.
# Header: X-JOBBER-GRAPHQL-VERSION: 2026-09-25
query VisitForCalendar($id: EncodedId!) {
  visit(id: $id) {
    __typename
    id
    title
    startAt          # null when unscheduled
    endAt            # null when unscheduled
    allDay
    instructions
    visitStatus
    isComplete
    job {            # Visit.job is Job! (non-null) — this is the Visit -> Job relationship
      __typename
      id
```

```
      jobNumber
      title
      jobberWebUri
    }
    client {
      id
      name
    }
    property {
      id
      address {
        street1
        street2
        city
        province
        postalCode
        country
      }
    }
    assignedUsers(first: 25) {
      nodes {
        id
        name {
          full
        }
      }
    }
  }
}
```

scripts/sign-webhook.mjs

```
#!/usr/bin/env node
// Sends a correctly signed fake Jobber webhook to a running worker (local `wrangler dev` or deployed).
// Usage:
//   JOBBER_CLIENT_SECRET=... node scripts/sign-webhook.mjs <topic> <itemId> <accountId> [url]
// Example:
//   node scripts/sign-webhook.mjs VISIT_UPDATE Z2lkOi8vSm9iYmVyL1Zpc2l0LzEyMw== <accountId>
http://localhost:8787/webhooks/jobber
import { createHmac } from "node:crypto";

const [topic, itemId, accountId, url = "http://localhost:8787/webhooks/jobber"] = process.argv.slice(2);
const secret = process.env.JOBBER_CLIENT_SECRET;
if (!topic || !itemId || !accountId || !secret) {
  console.error("Usage: JOBBER_CLIENT_SECRET=... node scripts/sign-webhook.mjs <topic> <itemId> <accountId>
[url]");
  process.exit(1);
}

const body = JSON.stringify({
  data: { webHookEvent: { topic, appId: "local-test", accountId, itemId, occurredAt: new Date().toISOString() }
},
});
const signature = createHmac("sha256", secret).update(body).digest("base64");

const res = await fetch(url, {
  method: "POST",
  headers: { "content-type": "application/json", "X-Jobber-Hmac-SHA256": signature },
  body,
```

```
});
console.log(res.status, await res.text());
```